

Lab1

July 26, 2026

0.0.1 Laboratorio 1 – Series de Tiempo

Análisis exploratorio — incisos d, e, f

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# El archivo .xlsx es realmente un CSV
df = pd.read_csv("./Datos/Base_Migracion_2009-2026jun.csv")
df['Viajero'] = pd.to_numeric(df['Viajero'], errors='coerce')

# Subconjunto consistente para todo el período 2009-2026
# entre 2022-2023 la categoría 'Viajero' excluyó viajeros no turísticos de alta
# frecuencia
# (comercio fronterizo, tránsito), por lo que solo Turista + Excursionista son
# comparables
df_te = df[df['Tipo de Viajero'].isin(['Turista', 'Excursionista'])].copy()

print(f"Total filas: {len(df)}, Turista+Excursionista: {len(df_te)}")
df.head(3)

df_filtracion = df[df['Tipo de Viajero'].isin(['Turista', 'Excursionista'])].
    copy()
df_filtracion['Fecha'] = pd.to_datetime(df_filtracion['Año'].astype(str) + '-' +
    df_filtracion['Mes cod'].astype(str) + '-01')
```

Total filas: 161036, Turista+Excursionista: 137642

0.0.2 a,b,c) comportamiento temporal del número de viajeros, países con mayor cantidad de viajeros. regiones con mayor cantidad de viajeros

```
[2]: #EXPLORACION INICIAL

print("\n INFORMACIÓN GENERAL")
print("-"*70)
print(f"Dimensiones: {df.shape[0]} filas × {df.shape[1]} columnas")
print(f"Años: {df['Año'].min()} - {df['Año'].max()}")
print(f"Tipos de viajero: {df['Tipo de Viajero'].unique().tolist()}")
```

```

print("\nTipos de df:")
print(df.dtypes)

print("\nValores nulos por columna:")
print(df.isnull().sum())

print("Estadística descriptiva viajeros:")
print(df['Viajero'].describe().apply(lambda x: f'{x:,.0f}'))

print("Categorías únicas:")
print(f"Vías: {df['Vía'].unique()}")
print(f"Tipos de viajero: {df['Tipo de Viajero'].unique()}")
print(f"Años: {df['Año'].unique()}")

#tendencias: mensual y anual
fig, axes = plt.subplots(2, 2, figsize=(15, 10))

mensual = df_filtracion.groupby('Fecha')['Viajero'].sum()
axes[0, 0].plot(mensual.index, mensual.values, color='blue', linewidth=1)
axes[0, 0].set_title('Evolución Mensual de Viajeros (Turista + Excursionista)',
    ↪fontsize=12)
axes[0, 0].set_xlabel('Fecha')
axes[0, 0].set_ylabel('Número de Viajeros')
axes[0, 0].grid(True, alpha=0.3)

anual = df_filtracion.groupby('Año')['Viajero'].sum()
anual.index = anual.index.astype(int)
axes[0, 1].bar(anual.index, anual.values, color='skyblue')
axes[0, 1].set_title('Total Anual de Viajeros (Turista + Excursionista)',
    ↪fontsize=12)
axes[0, 1].set_xlabel('Año')
axes[0, 1].set_ylabel('Número de Viajeros')
axes[0, 1].set_xticks(anual.index)
axes[0, 1].set_xticklabels(anual.index, rotation=45)
axes[0, 1].ticklabel_format(style='plain', axis='y', useOffset=False)
axes[0, 1].yaxis.set_major_formatter(plt.FuncFormatter(lambda x, p:
    ↪format(int(x))))

estacionalidad = df_filtracion.groupby('Mes')['Viajero'].mean()
meses_orden = ['Ene', 'Feb', 'Mar', 'Abr', 'May', 'Jun', 'Jul', 'Ago', 'Sep',
    ↪'Oct', 'Nov', 'Dic']
estacionalidad = estacionalidad.reindex(meses_orden)
axes[1, 0].plot(estacionalidad.index, estacionalidad.values, color='green')
axes[1, 0].set_title('Estacionalidad Mensual Promedio', fontsize=12)
axes[1, 0].set_xlabel('Mes')

```

```

axes[1, 0].set_ylabel('Promedio de Viajeros')
axes[1, 0].grid(True, alpha=0.3)

media_movil = mensual.rolling(window=12).mean()
axes[1, 1].plot(mensual.index, mensual.values, color='gray', alpha=0.5,
    ↳label='df mensuales')
axes[1, 1].plot(media_movil.index, media_movil.values, color='red',
    ↳linewidth=2, label='Media móvil 12 meses')
axes[1, 1].set_title('Tendencia con Media Móvil', fontsize=12)
axes[1, 1].set_xlabel('Fecha')
axes[1, 1].set_ylabel('Número de Viajeros')
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

#b. países con mayor cantidad de viajeros
top_paises = df_filtracion.groupby('País')['Viajero'].sum().
    ↳sort_values(ascending=False).head(10)
fig, ax = plt.subplots(figsize=(15, 8))
bars = ax.barh(top_paises.index, top_paises.values, color='coral',
    ↳edgecolor='darkred')
ax.set_title('Top 10 Países con Mayor Cantidad de Viajeros', fontsize=14)
ax.set_xlabel('Total de Viajeros', fontsize=12)
ax.invert_yaxis()

for i, (bar, value) in enumerate(zip(bars, top_paises.values)):
    ax.text(value, bar.get_y() + bar.get_height()/2,
        f' {int(value):,}', va='center', ha='left', fontsize=9)

plt.tight_layout()
plt.show()

#c. regiones con mayor cantidad de viajeros
top_regiones = df_filtracion.groupby('Región dos')['Viajero'].sum().
    ↳sort_values(ascending=False)

fig, ax = plt.subplots(figsize=(12, 8))

# Crear barras horizontales
bars = ax.barh(top_regiones.index, top_regiones.values, color=plt.cm.Set2(np.
    ↳linspace(0, 1, len(top_regiones))))

# Agregar valores y porcentajes al final de cada barra
total = top_regiones.sum()

```

```

for i, (bar, value) in enumerate(zip(bars, top_regiones.values)):
    pct = (value / total) * 100
    ax.text(value + (total * 0.01), bar.get_y() + bar.get_height()/2,
            f'{int(value):,} ({pct:.1f}%)',
            va='center', ha='left', fontsize=10)

ax.set_title('Distribución Viajeros por Región (Continentes)', fontsize=14,
            weight='bold')
ax.set_xlabel('Total de Viajeros', fontsize=12)
ax.set_ylabel('Región', fontsize=12)
ax.ticklabel_format(style='plain', axis='x')
ax.xaxis.set_major_formatter(plt.FuncFormatter(lambda x, p: format(int(x),
            ', ')))

plt.tight_layout()
plt.show()

```

INFORMACIÓN GENERAL

Dimensiones: 161036 filas × 13 columnas

Años: 2009 - 2026

Tipos de viajero: ['Turista', 'Cruceiristas', 'Viajero', 'Excursionista']

Tipos de df:

Año	int64
Mes cod	int64
Mes	str
Vía	str
Frontera	str
País	str
Región	str
Región dos	str
Regiones OMT	str
MCEO	str
Agrupación Residencia	str
Tipo de Viajero	str
Viajero	float64
dtype:	object

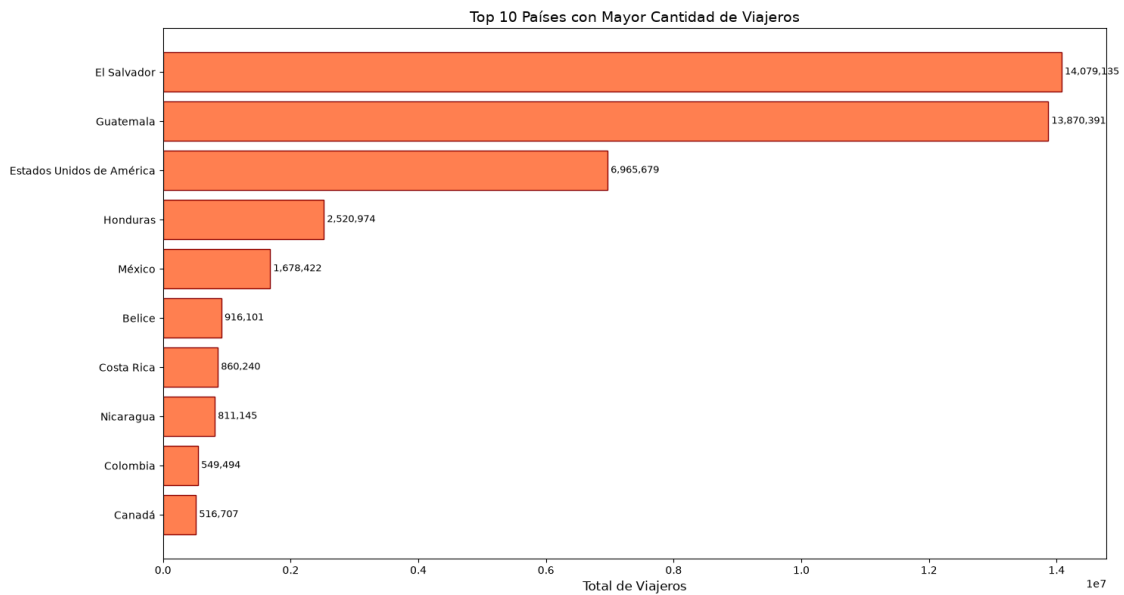
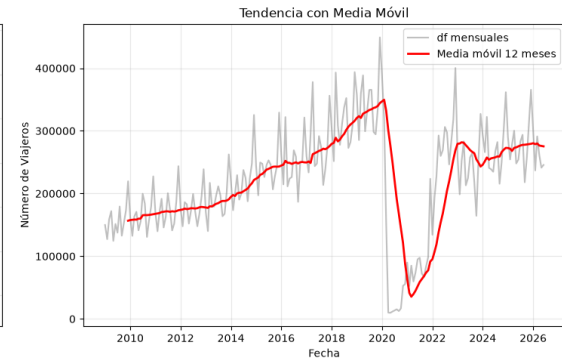
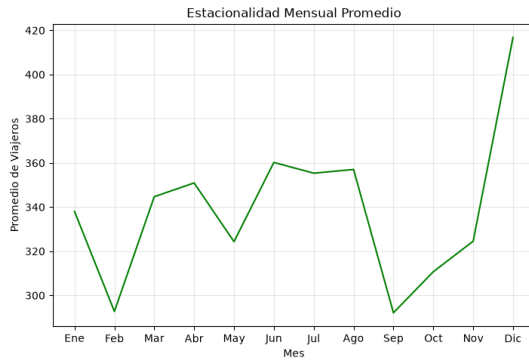
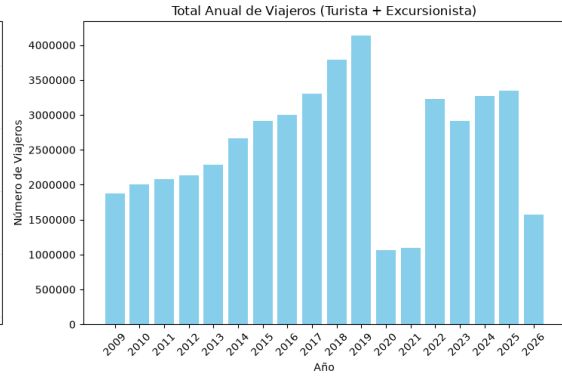
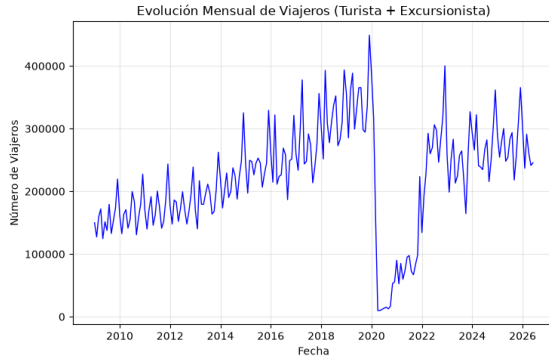
Valores nulos por columna:

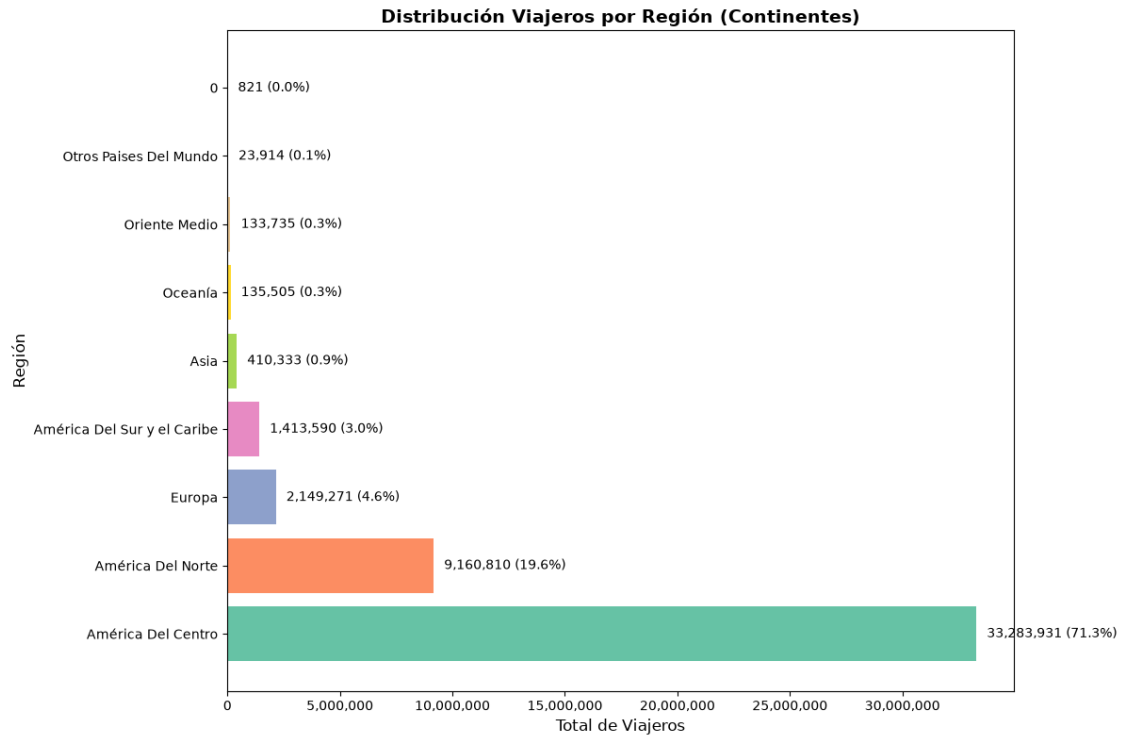
Año	0
Mes cod	0
Mes	0
Vía	0
Frontera	0
País	0

```

Región          0
Región dos      0
Regiones OMT    0
MCEO            0
Agrupación Residencia  0
Tipo de Viajero  0
Viajero         0
dtype: int64
Estadística descriptiva viajeros:
count    161,036
mean       325
std        2,388
min         0
25%         2
50%         7
75%        39
max       92,336
Name: Viajero, dtype: str
Categorías únicas:
Vias: <StringArray>
['Aérea', 'Marítima', 'Terrestre']
Length: 3, dtype: str
Tipos de viajero: <StringArray>
['Turista', 'Cruceiristas', 'Viajero', 'Excursionista']
Length: 4, dtype: str
Años: [2009 2010 2011 2012 2013 2014 2015 2016 2017 2018 2019 2020 2021 2022
       2023 2024 2025 2026]

```





0.0.3 d) Vías de ingreso y fronteras más utilizadas

```
[3]: # Vías de ingreso por volumen de viajeros
vias = (
    df_te.groupby("Vía")["Viajero"]
    .sum()
    .sort_values(ascending=False)
)
vias_pct = 100 * vias / vias.sum()
print("=== VÍAS DE INGRESO POR VOLUMEN DE VIAJEROS ===")
display(pd.DataFrame({"Viajeros": vias, "Porcentaje": vias_pct})
    .style.format({"Viajeros": "{:,.0f}", "Porcentaje": "{:.2f}%"}))

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
vias.plot(kind="bar", ax=axes[0], color=["salmon", "skyblue", "lightgreen"])
axes[0].set_title("Viajeros por vía de ingreso")
axes[0].set_ylabel("Total de viajeros")
axes[0].tick_params(axis="x", rotation=0)
vias.plot(
    kind="pie", ax=axes[1], autopct="%1.1f%%", startangle=90,
    colors=["salmon", "skyblue", "lightgreen"],
    wedgeprops={"edgecolor": "black"},
)
```

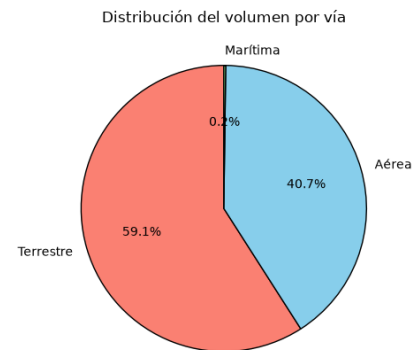
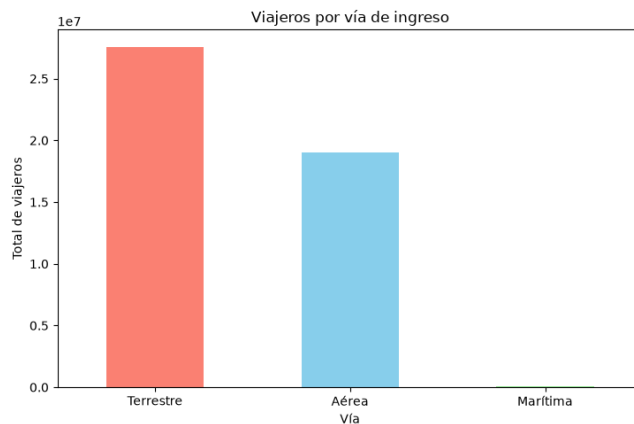
```

axes[1].set_title("Distribución del volumen por vía")
axes[1].set_ylabel("")
plt.tight_layout()
plt.show()

```

=== VÍAS DE INGRESO POR VOLUMEN DE VIAJEROS ===

<pandas.io.formats.style.Styler at 0x11da999a0>



```

[4]: # Fronteras por volumen de viajeros
fronteras = (
    df_te.groupby("Frontera")["Viajero"]
    .sum()
    .sort_values(ascending=False)
)
print("=== TOP 10 FRONTERAS POR VOLUMEN DE VIAJEROS ===")
display(fronteras.head(10).to_frame("Viajeros")
        .style.format({"Viajeros": "{:,.0f}"}))
print(
    f"Porcentaje acumulado del top 5: "
    f"{100 * fronteras.head(5).sum() / fronteras.sum():.2f}%"
)

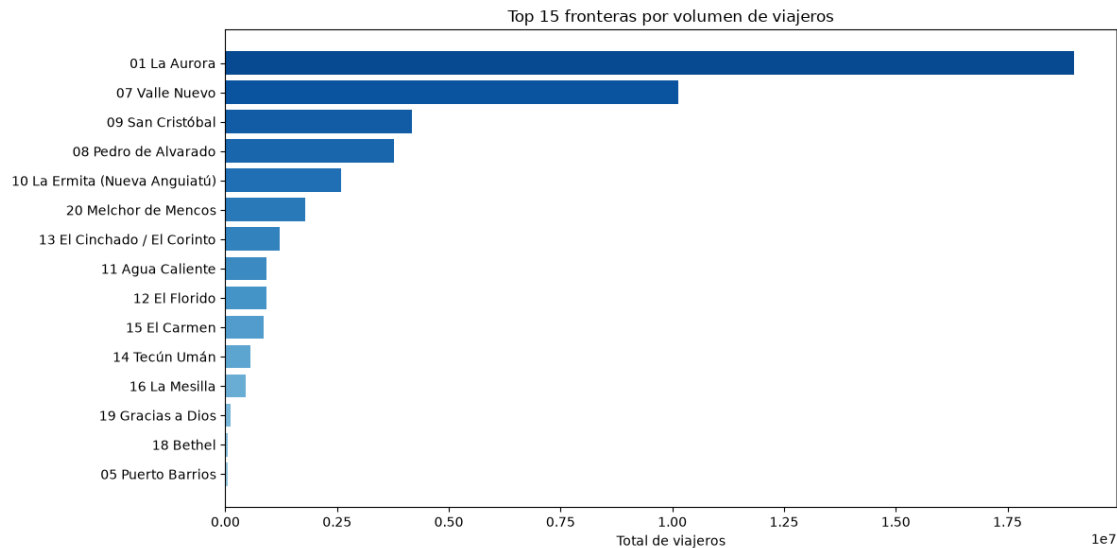
fig, ax = plt.subplots(figsize=(12, 6))
top15 = fronteras.head(15)
ax.barh(top15.index, top15.values, color=plt.cm.Blues(np.linspace(.4, .9, 15))[:
↪ :-1])
ax.invert_yaxis()
ax.set_title("Top 15 fronteras por volumen de viajeros")
ax.set_xlabel("Total de viajeros")
plt.tight_layout()
plt.show()

```

=== TOP 10 FRONTERAS POR VOLUMEN DE VIAJEROS ===

<pandas.io.formats.style.Styler at 0x11a4cc590>

Porcentaje acumulado del top 5: 84.92%



Interpretación:

- La vía **terrestre** concentra aproximadamente 59.1% del volumen de Turista + Excursionista; la aérea representa 40.7% y la marítima 0.2%.
- **La Aurora** es la frontera individual con mayor volumen, seguida de Valle Nuevo y San Cristóbal.
- Las cinco fronteras principales concentran aproximadamente 84.9% de los viajeros comparables. Esta concentración es útil para priorizar capacidad operativa, pero también aumenta la exposición a interrupciones puntuales.

0.0.4 e) Análisis de valores faltantes, duplicados y valores atípicos

```
[5]: # Valores faltantes
print("=== VALORES FALTANTES ===")
missing = df.isnull().sum()
missing_pct = (df.isnull().sum() / len(df) * 100).round(4)
missing_info = pd.DataFrame({'Faltantes': missing, 'Porcentaje': missing_pct})
print(missing_info[missing_info['Faltantes'] > 0] if missing.sum() > 0 else "No
    ↪hay valores faltantes en ninguna columna.")

# Duplicados
dups = df.duplicated().sum()
print(f"\n=== DUPLICADOS ===")
print(f"Filas exactamente duplicadas: {dups}")
```

```

# Valores atípicos en Viajero (IQR)
print(f"\n=== VALORES ATÍPICOS (Viajero) ===")
Q1 = df['Viajero'].quantile(0.25)
Q3 = df['Viajero'].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
outliers = df[(df['Viajero'] < lower) | (df['Viajero'] > upper)]
print(f"Q1 = {Q1:.2f}, Q3 = {Q3:.2f}, IQR = {IQR:.2f}")
print(f"Límite inferior = {lower:.2f}, Límite superior = {upper:.2f}")
print(f"Registros atípicos: {len(outliers)} ({len(outliers)/len(df)*100:.2f}%  

    ↪del total)")

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Boxplot
axes[0].boxplot(df['Viajero'].clip(upper=df['Viajero'].quantile(0.99)),  

    ↪vert=False)
axes[0].set_title('Boxplot de Viajeros (recortado al percentil 99)')
axes[0].set_xlabel('Viajeros')

# Histograma
df['Viajero'].clip(upper=df['Viajero'].quantile(0.99)).hist(bins=50, ax=axes[1],  

    color='coral', edgecolor='black', alpha=0.7)
axes[1].set_title('Distribución de Viajeros (percentil 99)')
axes[1].set_xlabel('Viajeros')
axes[1].set_ylabel('Frecuencia')

plt.tight_layout()
plt.show()

```

=== VALORES FALTANTES ===

No hay valores faltantes en ninguna columna.

=== DUPLICADOS ===

Filas exactamente duplicadas: 0

=== VALORES ATÍPICOS (Viajero) ===

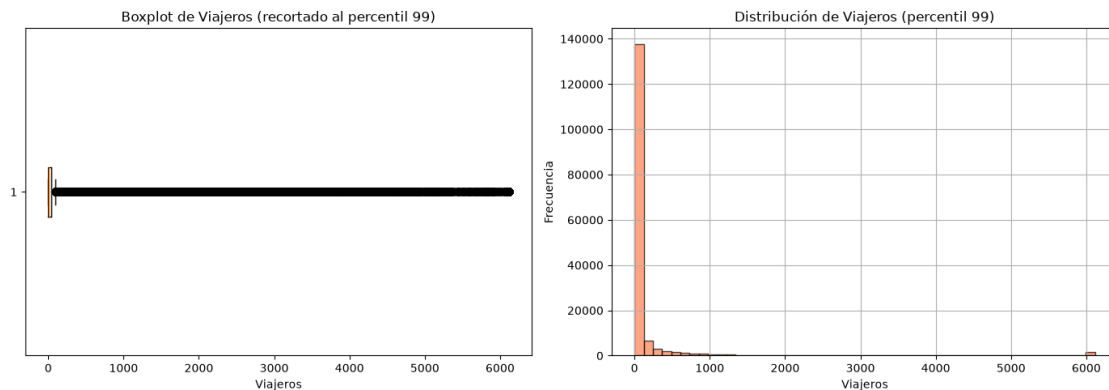
Q1 = 2.00, Q3 = 38.89, IQR = 36.89

Límite inferior = -53.34, Límite superior = 94.23

Registros atípicos: 26390 (16.39% del total)

/var/folders/jq/zj1d7dc15dd57tj80xs_lvxr0000gn/T/ipykernel_15964/900652176.py:28
: MatplotlibDeprecationWarning: vert: bool was deprecated in Matplotlib 3.11 and
will be removed in 3.13. Use orientation: {'vertical', 'horizontal'} instead.
 axes[0].boxplot(df['Viajero'].clip(upper=df['Viajero'].quantile(0.99)),

```
vert=False)
```



Interpretación:

- El conjunto de df **no presenta valores faltantes ni filas duplicadas**, indicando que la base está completa y limpia.
- Sin embargo, ~16.4% de los registros se identifican como **valores atípicos** en 'Viajero' según el criterio IQR. Esto es esperable en df de migración: ciertas rutas/fronteras concentran volúmenes muy superiores al promedio.
- La distribución está **fuertemente sesgada a la derecha**: la mayoría de observaciones tienen pocos viajeros (mediana = 7), pero existen picos muy altos que elevan la media a ~325.

0.0.5 f) Estadísticas descriptivas y visualizaciones con interpretación

```
[6]: # Estadísticas descriptivas
print("=== ESTADÍSTICAS DESCRIPTIVAS (variable Viajero) ===")
desc = df['Viajero'].describe()
print(desc)
print(f"\nVarianza: {df['Viajero'].var():.2f}")
print(f"Sesgo (skewness): {df['Viajero'].skew():.2f}")
print(f"Curtosis: {df['Viajero'].kurtosis():.2f}")

# Estadísticas por vía de ingreso
print("\n=== ESTADÍSTICAS POR VÍA DE INGRESO ===")
print(df.groupby('Vía')['Viajero'].describe().round(2))

# Estadísticas por tipo de viajero
print("\n=== ESTADÍSTICAS POR TIPO DE VIAJERO ===")
print(df.groupby('Tipo de Viajero')['Viajero'].describe().round(2))

=== ESTADÍSTICAS DESCRIPTIVAS (variable Viajero) ===
count      161036.000000
mean         324.697193
std         2387.745140
min           0.000000
```

```

25%          2.000000
50%          7.000000
75%         38.891667
max         92336.035067
Name: Viajero, dtype: float64

```

```

Varianza: 5701326.85
Sesgo (skewness): 15.44
Curtosis: 298.01

```

=== ESTADÍSTICAS POR VÍA DE INGRESO ===

	count	mean	std	min	25%	50%	75%	max
Vía								
Aérea	24999.0	762.58	4589.39	0.0	2.0	7.00	125.00	83511.00
Marítima	10164.0	120.90	964.98	1.0	1.0	3.00	11.00	21680.93
Terrestre	125873.0	254.19	1729.00	0.0	2.0	7.44	36.85	92336.04

=== ESTADÍSTICAS POR TIPO DE VIAJERO ===

	count	mean	std	min	25%	50%	75%	\
Tipo de Viajero								
Cruceristas	204.0	5413.74	4225.76	85.0	2094.0	4454.91	7173.50	
Excursionista	19730.0	459.66	1833.67	0.0	2.0	10.00	80.26	
Turista	117912.0	319.24	2611.32	0.0	2.0	6.00	33.00	
Viajero	23190.0	192.83	1281.23	0.0	2.0	8.18	44.00	

	max
Tipo de Viajero	
Cruceristas	21680.93
Excursionista	38463.03
Turista	83511.00
Viajero	92336.04

```

[7]: # Visualizaciones (usando Turista+Excursionista para consistencia longitudinal)
fig, axes = plt.subplots(2, 2, figsize=(16, 10))

# 1. Serie de tiempo mensual (Turista+Excursionista)
serie = df_te.groupby(['Año', 'Mes cod'])['Viajero'].sum().reset_index()
serie['fecha'] = pd.to_datetime(serie['Año'].astype(str) + '-' + serie['Mes_
    ↪cod'].astype(str))
serie = serie.sort_values('fecha')
axes[0, 0].plot(serie['fecha'], serie['Viajero'], color='steelblue',
    ↪linewidth=1)
axes[0, 0].set_title('Evolución Mensual - Turista+Excursionista')
axes[0, 0].set_xlabel('Fecha')
axes[0, 0].set_ylabel('Viajeros')
axes[0, 0].tick_params(axis='x', rotation=45)

```

```

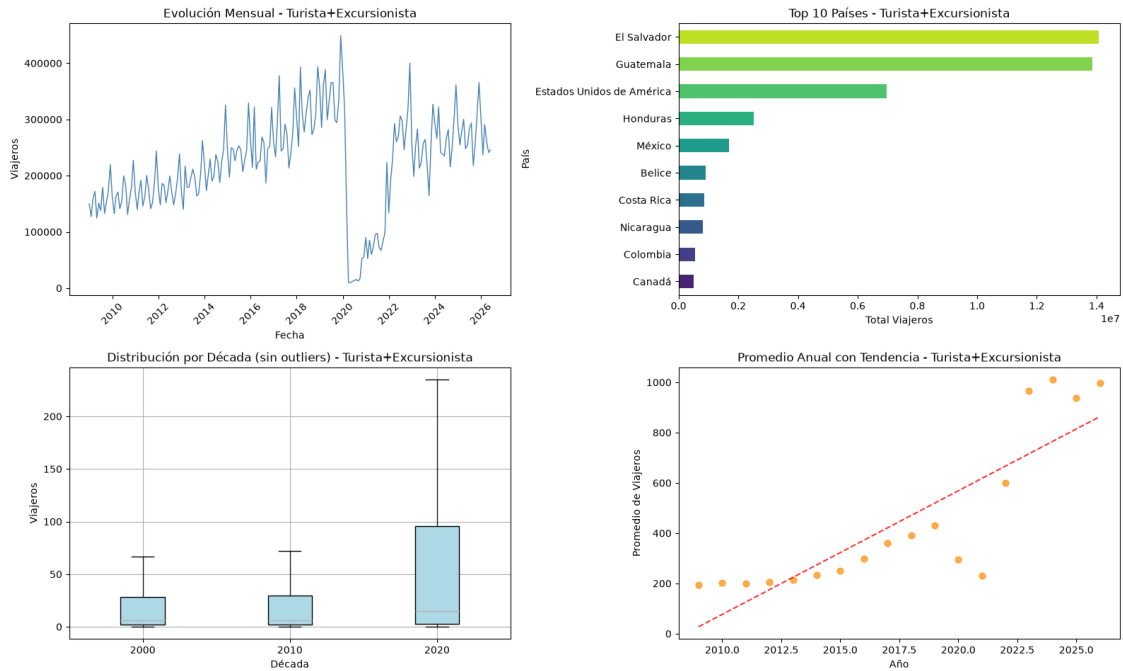
# 2. Top 10 países (Turista+Excursionista)
top_paises = df_te.groupby('País')['Viajero'].sum().
    ↪sort_values(ascending=False).head(10)
colors = plt.cm.viridis(np.linspace(0.1, 0.9, 10))
top_paises.plot(kind='barh', ax=axes[0, 1], color=colors[::-1])
axes[0, 1].set_title('Top 10 Países - Turista+Excursionista')
axes[0, 1].set_xlabel('Total Viajeros')
axes[0, 1].invert_yaxis()

# 3. Distribución por década (Turista+Excursionista)
df_te['década'] = (df_te['Año'] // 10) * 10
df_te.boxplot(column='Viajero', by='década', ax=axes[1, 0],
               showfliers=False, patch_artist=True,
               boxprops=dict(facecolor='lightblue'))
axes[1, 0].set_title('Distribución por Década (sin outliers) - ↵
    ↪Turista+Excursionista')
axes[1, 0].set_xlabel('Década')
axes[1, 0].set_ylabel('Viajeros')

# 4. Promedio anual (Turista+Excursionista)
anual = df_te.groupby('Año')['Viajero'].mean()
axes[1, 1].scatter(anual.index, anual.values, color='darkorange', alpha=0.7, ↵
    ↪s=40)
z = np.polyfit(anual.index, anual.values, 1)
p = np.poly1d(z)
axes[1, 1].plot(anual.index, p(anual.index), 'r--', alpha=0.8)
axes[1, 1].set_title('Promedio Anual con Tendencia - Turista+Excursionista')
axes[1, 1].set_xlabel('Año')
axes[1, 1].set_ylabel('Promedio de Viajeros')

plt.suptitle('Análisis Exploratorio - Visualizaciones (Turista+Excursionista)', ↵
    ↪fontsize=14, y=1.02)
plt.tight_layout()
plt.show()

```



Interpretación general:

1. La serie Turista + Excursionista muestra crecimiento antes de 2020, una caída abrupta durante la pandemia y una recuperación posterior.
2. La distribución por registro es altamente asimétrica: existen muchos valores pequeños y pocos volúmenes muy grandes.
3. Por volumen, la vía terrestre aporta cerca de 59.1% y la aérea 40.7%; la vía marítima representa solo 0.2% bajo el filtro comparable.
4. La Aurora lidera el volumen por frontera, mientras que el conjunto de pasos terrestres concentra la mayor participación.
5. Entre 2022 y 2023 cambió la definición de la categoría Viajero. Por ello los análisis longitudinales usan Turista + Excursionista.

1 2. División temporal y construcción de las series

Para evitar fuga de información, la división 70/30 se realiza **después de agregar por mes** y sin mezclar observaciones futuras en entrenamiento. Siguiendo la advertencia del enunciado, todos los análisis longitudinales usan únicamente **Turista + Excursionista**, categorías comparables durante 2009-2026. Los meses ausentes se completan con cero.

```
[8]: from itertools import product
import warnings

import pandas as pd
```

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import linregress
from statsmodels.tsa.seasonal import STL
from statsmodels.tsa.stattools import adfuller, acf, pacf
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.holtwinters import ExponentialSmoothing, SimpleExpSmoothing
from statsmodels.stats.diagnostic import acorr_ljungbox
from sklearn.metrics import mean_absolute_error, mean_squared_error
from prophet import Prophet

warnings.filterwarnings("ignore")
plt.style.use("seaborn-v0_8-whitegrid")

# Base comparable y eje mensual común
df_ts = df[df["Tipo de Viajero"].isin(["Turista", "Excursionista"])] .copy()
df_ts["Fecha"] = pd.to_datetime(
    dict(year=df_ts["Año"], month=df_ts["Mes cod"], day=1)
)
calendario = pd.date_range(df_ts["Fecha"].min(), df_ts["Fecha"].max(),
    freq="MS")

def serie_mensual(datos, filtro_col=None, filtro_val=None):
    subset = datos if filtro_col is None else datos[datos[filtro_col] ==
    filtro_val]
    return (
        subset.groupby("Fecha")["Viajero"].sum()
        .reindex(calendario, fill_value=0)
        .astype(float)
        .asfreq("MS")
    )

def dividir_serie(serie, proporcion=0.70):
    corte = int(np.floor(len(serie) * proporcion))
    return serie.iloc[:corte].copy(), serie.iloc[corte:].copy()

serie_total = serie_mensual(df_ts)
total_train, total_test = dividir_serie(serie_total)

resumen_division = pd.DataFrame(
    {
        "Conjunto": ["Serie completa", "Entrenamiento", "Prueba"],
        "Inicio": [
            serie_total.index.min(),
            total_train.index.min(),
            total_test.index.min(),
        ]
    }
)

```

```

    ],
    "Fin": [
        serie_total.index.max(),
        total_train.index.max(),
        total_test.index.max(),
    ],
    "Meses": [len(serie_total), len(total_train), len(total_test)],
    "Porcentaje": [
        100,
        100 * len(total_train) / len(serie_total),
        100 * len(total_test) / len(serie_total),
    ],
}
)
display(resumen_division.style.format({"Porcentaje": "{:.1f}%"})
print("Frecuencia:", serie_total.index.freqstr)

```

```

/Users/hansellopez/Documents/Projects/DataScience2026/Labs_DataSci/Lab
1/.venv/lib/python3.12/site-packages/tqdm/auto.py:21: TqdmWarning: IProgress not
found. Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
    from .autonotebook import tqdm as notebook_tqdm
Importing plotly failed. Interactive plots will not work.

<pandas.io.formats.style.Styler at 0x12ae58aa0>

Frecuencia: MS

```

2 3a. Serie obligatoria: total mensual de viajeros internacionales

2.1 4a-4d. Inspección, descomposición y transformación

La necesidad de transformar se evalúa con la relación entre media y varianza móviles. Además, se estudia $\log(1 + \text{viajeros})$ como escala de modelado robusta: admite ceros, reduce la influencia del choque de 2020 y evita predicciones negativas al regresar a la escala original. La fuerza estacional se cuantifica como $(1 - \text{Var}(R) / \text{Var}(S+R))$, acotada entre 0 y 1.

```

[9]: fig, ax = plt.subplots(figsize=(15, 5))
ax.plot(serie_total, color="navy", lw=1.6, label="Total mensual")
ax.axvspan(pd.Timestamp("2020-03-01"), pd.Timestamp("2020-12-01"),
           color="firebrick", alpha=.12, label="Pandemia 2020")
ax.axvline(total_test.index[0], color="black", ls="--", label="Inicio de
prueba")
ax.set(title="Serie obligatoria: Turista + Excursionista",
       xlabel="Fecha", ylabel="Viajeros")
ax.legend()
plt.show()

# Evidencia para decidir transformación

```

```

medias_12 = total_train.rolling(12).mean().dropna()
varianzas_12 = total_train.rolling(12).var().dropna()
corr_media_var = medias_12.corr(varianzas_12)
print(f"Correlación entre media y varianza móviles (12 meses): {corr_media_var:.3f}")

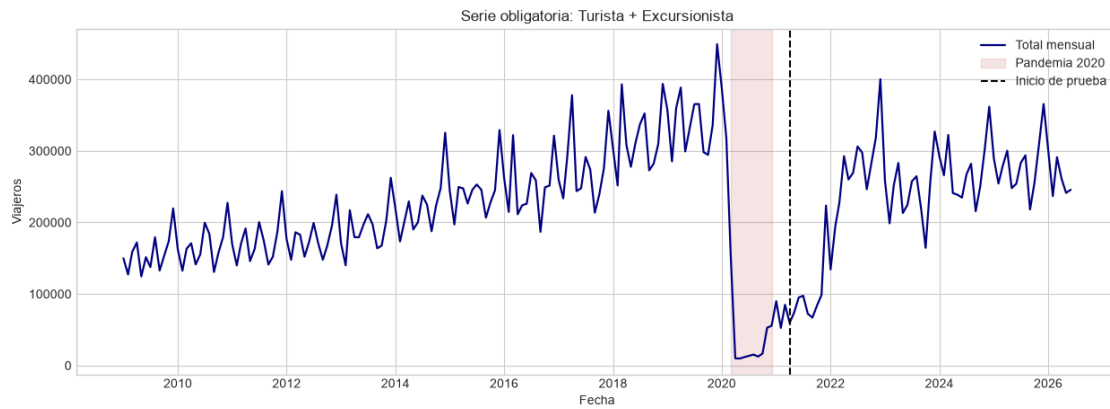
if corr_media_var > 0.5:
    print("Existe evidencia de varianza creciente con el nivel: conviene_
    ↪transformar.")
else:
    print(
        "No hay evidencia fuerte de que la transformación sea estrictamente "
        "necesaria para la varianza."
    )
print(
    "Para el modelado se usa loglp como decisión robusta frente al choque de_
    ↪2020 "
    "y para obtener pronósticos no negativos; los errores se reportan en_
    ↪viajeros."
)

# La descomposición usada para diagnóstico/modelado se estima solamente
# con entrenamiento. La gráfica completa anterior se conserva para la
# descripción histórica y de pandemia.
log_total_train = np.loglp(total_train)
stl_total = STL(log_total_train, period=12, robust=True).fit()
fuerza_est_total = max(
    0.0,
    1 - np.var(stl_total.resid) / np.var(stl_total.seasonal + stl_total.resid),
)

fig = stl_total.plot()
fig.set_size_inches(15, 9)
fig.suptitle(
    "Descomposición STL de log(1 + total mensual) - entrenamiento", y=1.01
)
plt.show()
print(f"Fuerza estacional (0-1): {fuerza_est_total:.3f}")
intensidad = "débil" if fuerza_est_total < 0.3 else "moderada o fuerte"
print(
    f"Lectura: la estacionalidad estimada es {intensidad}; la tendencia cambia "
    "fuertemente alrededor de 2020, por lo que la serie original no es "
    "estacionaria en media."
)
impacto_total = 100 * (serie_total.loc["2020"].sum() / serie_total.loc["2019"].
    ↪sum() - 1)
recuperacion_total = 100 * (serie_total.loc["2025"].sum() / serie_total.
    ↪loc["2019"].sum() - 1)

```

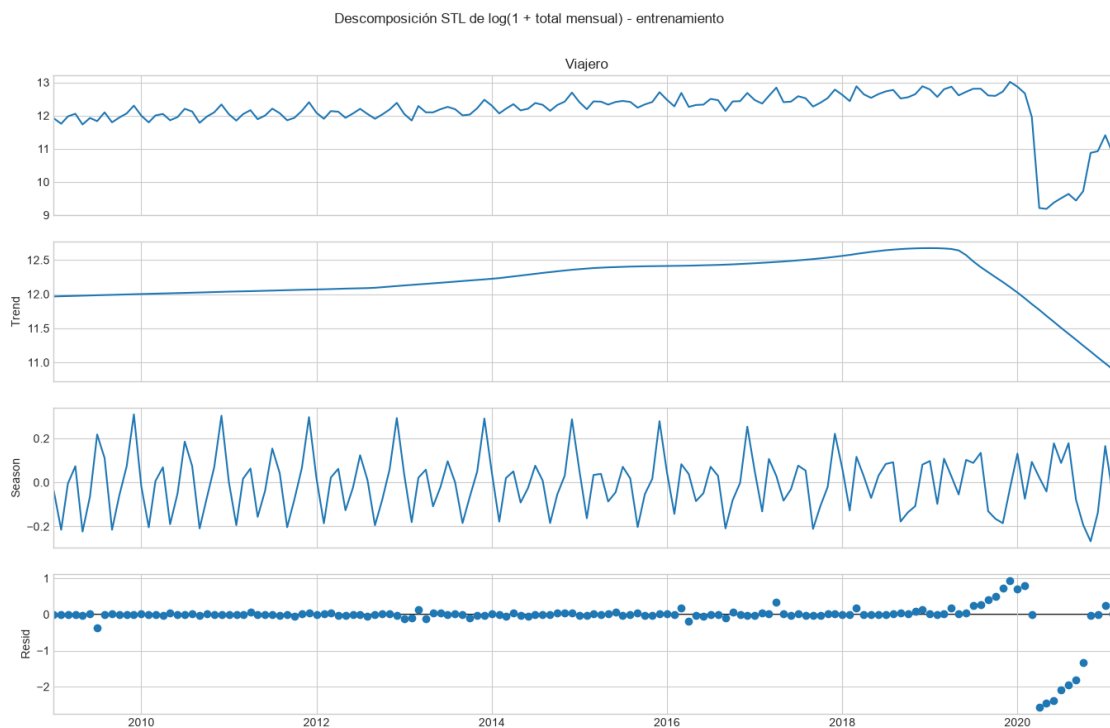
```
print(f"Impacto 2020 frente a 2019: {impacto_total:+.1f}%.")
print(f"Nivel 2025 frente a 2019: {recuperacion_total:+.1f}%.")
```



Correlación entre media y varianza móviles (12 meses): -0.080

No hay evidencia fuerte de que la transformación sea estrictamente necesaria para la varianza.

Para el modelado se usa `log1p` como decisión robusta frente al choque de 2020 y para obtener pronósticos no negativos; los errores se reportan en viajeros.



Fuerza estacional (0-1): 0.030

Lectura: la estacionalidad estimada es débil; la tendencia cambia fuertemente alrededor de 2020, por lo que la serie original no es estacionaria en media. Impacto 2020 frente a 2019: -74.4%. Nivel 2025 frente a 2019: -18.9%.

2.2 4e-4f. Estacionariedad y elección de p, d, q

Se usa ADF con nivel de significancia de 5%. El número de diferencias d es el mínimo que rechaza la hipótesis nula de raíz unitaria. Los gráficos ACF/PACF de la serie transformada y diferenciada orientan los órdenes q y p ; en vez de afirmar un único corte visual ambiguo, se evalúa una cuadrícula pequeña alrededor de órdenes 0-2 y se selecciona por AIC/BIC.

```
[10]: def tabla_adf(serie, max_d=2):
    filas = []
    actual = serie.dropna().copy()
    for d in range(max_d + 1):
        resultado = adfuller(actual, autolag="AIC")
        filas.append(
            {
                "d": d,
                "ADF": resultado[0],
                "p_value": resultado[1],
                "rezagos": resultado[2],
                "n": resultado[3],
                "estacionaria_5pct": resultado[1] < 0.05,
            }
        )
        if resultado[1] < 0.05:
            break
        actual = actual.diff().dropna()
    return pd.DataFrame(filas), actual

adf_total, total_estacionaria = tabla_adf(np.log1p(total_train))
display(adf_total.style.format({"ADF": "{:.3f}", "p_value": "{:.4f}"}))
d_total = int(adf_total.iloc[-1]["d"])

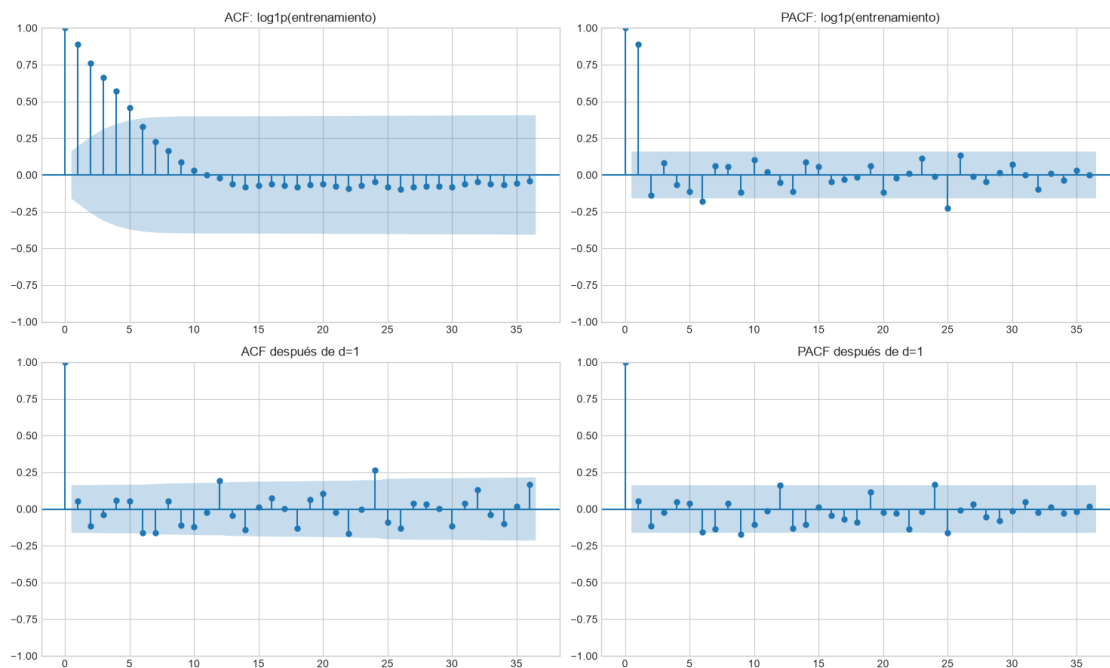
fig, axes = plt.subplots(2, 2, figsize=(15, 9))
plot_acf(np.log1p(total_train), lags=36, ax=axes[0, 0])
plot_pacf(np.log1p(total_train), lags=36, ax=axes[0, 1], method="yw")
plot_acf(total_estacionaria, lags=36, ax=axes[1, 0])
plot_pacf(total_estacionaria, lags=36, ax=axes[1, 1], method="yw")
axes[0, 0].set_title("ACF: log1p(entrenamiento)")
axes[0, 1].set_title("PACF: log1p(entrenamiento)")
axes[1, 0].set_title(f"ACF después de d={d_total}")
axes[1, 1].set_title(f"PACF después de d={d_total}")
plt.tight_layout()
plt.show()
```

```

limite = 1.96 / np.sqrt(len(total_estacionaria))
acf_vals = acf(total_estacionaria, nlags=12, fft=True)
pacf_vals = pacf(total_estacionaria, nlags=12, method="yw")
sig_acf = [i for i in range(1, 6) if abs(acf_vals[i]) > limite]
sig_pacf = [i for i in range(1, 6) if abs(pacf_vals[i]) > limite]
print(f"d seleccionado por ADF: {d_total}")
print(f"Rezagos ACF significativos entre 1 y 5: {sig_acf or 'ninguno'}")
print(f"Rezagos PACF significativos entre 1 y 5: {sig_pacf or 'ninguno'}")
print(
    "Se contrastan p,q en {0,1,2}; esto cubre los primeros cortes de ACF/PACF "
    "sin sobreparametrizar una muestra mensual de entrenamiento."
)

```

<pandas.io.formats.style.Styler at 0x12e2133b0>



d seleccionado por ADF: 1

Rezagos ACF significativos entre 1 y 5: ninguno

Rezagos PACF significativos entre 1 y 5: ninguno

Se contrastan p,q en {0,1,2}; esto cubre los primeros cortes de ACF/PACF sin sobreparametrizar una muestra mensual de entrenamiento.

2.3 4g-4k. ARIMA, modelos alternativos, predicción y selección

Todos los modelos se ajustan **solo con entrenamiento**. Para ARIMA, AIC y BIC comparan modelos con la misma variable transformada; MAE y RMSE se calculan en la escala original sobre prueba. AIC/BIC no se usan para comparar ARIMA contra Prophet o suavizamientos, porque sus

verosimilitudes no son directamente equivalentes. La selección final prioriza RMSE y usa MAE como evidencia complementaria.

```
[11]: y_train_log = np.log1p(total_train)
candidatos = [
    (p, d_total, q)
    for p, q in product(range(3), range(3))
    if not (p == 0 and q == 0)
]
filas_arima = []
ajustes_arima = {}
pred_arima = {}

for orden in candidatos:
    try:
        ajuste = ARIMA(
            y_train_log,
            order=orden,
            enforce_stationarity=False,
            enforce_invertibility=False,
        ).fit()
        pron_log = ajuste.forecast(steps=len(total_test))
        pron = pd.Series(
            np.maximum(0, np.exp1(np.asarray(pron_log))),
            index=total_test.index,
        )
        lb_p = acorr_ljungbox(
            ajuste.resid.dropna(), lags=[12], return_df=True
        )["lb_pvalue"].iloc[0]
        filas_arima.append(
            {
                "Modelo": f"ARIMA{orden}",
                "AIC": ajuste.aic,
                "BIC": ajuste.bic,
                "Ljung_Box_p12": lb_p,
                "MAE_prueba": mean_absolute_error(total_test, pron),
                "RMSE_prueba": np.sqrt(mean_squared_error(total_test, pron)),
            }
        )
        ajustes_arima[orden] = ajuste
        pred_arima[orden] = pron
    except Exception as exc:
        print(f"No convergió ARIMA{orden}: {exc}")

tabla_arima = pd.DataFrame(filas_arima).sort_values("AIC").
    ↪reset_index(drop=True)
tabla_arima["Residuos_blanco_5pct"] = tabla_arima["Ljung_Box_p12"] > 0.05
```

```

display(
    tabla_arima.style.format(
        {
            "AIC": "{:.1f}",
            "BIC": "{:.1f}",
            "Ljung_Box_p12": "{:.3f}",
            "MAE_prueba": "{:,.0f}",
            "RMSE_prueba": "{:,.0f}",
        }
    )
)

candidatos_validos = tabla_arima[tabla_arima["Residuos_blanco_5pct"]]
if candidatos_validos.empty:
    fila_best_arima = tabla_arima.iloc[0]
    criterio_arima = "menor AIC; ningún candidato pasó Ljung-Box"
else:
    fila_best_arima = candidatos_validos.sort_values(["AIC", "BIC"]).iloc[0]
    criterio_arima = "menor AIC entre modelos con Ljung-Box p>0.05"
mejor_nombre_arima = fila_best_arima["Modelo"]
mejor_orden_arima = next(
    orden for orden in ajustes_arima if f"ARIMA{orden}" == mejor_nombre_arima
)
mejor_ajuste_arima = ajustes_arima[mejor_orden_arima]
mejor_pred_arima = pred_arima[mejor_orden_arima]
print(
    f"ARIMA seleccionado ({criterio_arima}): {mejor_nombre_arima}; "
    f"AIC={mejor_ajuste_arima.aic:.1f}, BIC={mejor_ajuste_arima.bic:.1f}."
)
print(
    "Ljung-Box p>0.05 implica que no queda autocorrelación estadísticamente "
    "detectable al rezago 12; p<=0.05 señala estructura residual pendiente."
)

mejor_ajuste_arima.plot_diagnostics(figsize=(14, 8))
plt.suptitle(f"Diagnóstico de residuos: {mejor_nombre_arima}", y=1.02)
plt.tight_layout()
plt.show()

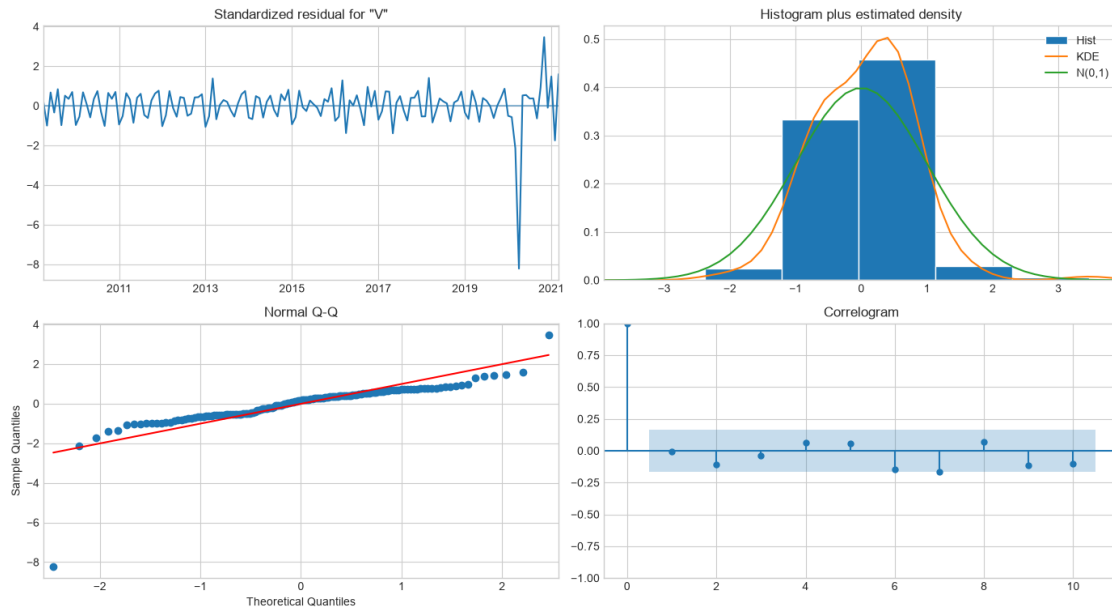
```

<pandas.io.formats.style.Styler at 0x12e432510>

ARIMA seleccionado (menor AIC entre modelos con Ljung-Box p>0.05): ARIMA(0, 1, 1); AIC=90.8, BIC=96.7.

Ljung-Box p>0.05 implica que no queda autocorrelación estadísticamente detectable al rezago 12; p<=0.05 señala estructura residual pendiente.

Diagnóstico de residuos: ARIMA(0, 1, 1)



```
[12]: predicciones = {mejor_nombre_arima: mejor_pred_arima}
      filas_modelos = []

      filas_modelos.append(
          {
              "Modelo": mejor_nombre_arima,
              "MAE": fila_best_arima["MAE_prueba"],
              "RMSE": fila_best_arima["RMSE_prueba"],
              "AIC": fila_best_arima["AIC"],
              "BIC": fila_best_arima["BIC"],
          }
      )

      # Holt-Winters con tendencia amortiguada y estacionalidad aditiva
      hw_fit = ExponentialSmoothing(
          total_train,
          trend="add",
          damped_trend=True,
          seasonal="add",
          seasonal_periods=12,
          initialization_method="estimated",
      ).fit(optimized=True)
      predicciones["Holt-Winters"] = hw_fit.forecast(len(total_test)).clip(lower=0)

      # Suavizamiento exponencial simple
```

```

ses_fit = SimpleExpSmoothing(
    total_train, initialization_method="estimated"
).fit(optimized=True)
predicciones["Suavizamiento exponencial"] = ses_fit.forecast(len(total_test)).
    ↪clip(lower=0)

# Seasonal naïve: repite cada mes el último valor observado del mismo mes
patron_12 = total_train.iloc[-12:].to_numpy()
predicciones["Seasonal naïve"] = pd.Series(
    np.resize(patron_12, len(total_test)), index=total_test.index
)

# Prophet con tendencia por tramos y estacionalidad anual
prophet_train = total_train.rename_axis("ds").reset_index(name="y")
prophet_model = Prophet(
    yearly_seasonality=True,
    weekly_seasonality=False,
    daily_seasonality=False,
    seasonality_mode="additive",
    changepoint_prior_scale=0.10,
    interval_width=0.95,
)
prophet_model.fit(prophet_train)
prophet_future = pd.DataFrame({"ds": total_test.index})
prophet_fcst = prophet_model.predict(prophet_future)
predicciones["Prophet"] = pd.Series(
    np.maximum(0, prophet_fcst["yhat"].to_numpy()), index=total_test.index
)

for nombre, pron in predicciones.items():
    if nombre == mejor_nombre_arima:
        continue
    filas_modelos.append(
        {
            "Modelo": nombre,
            "MAE": mean_absolute_error(total_test, pron),
            "RMSE": np.sqrt(mean_squared_error(total_test, pron)),
            "AIC": np.nan,
            "BIC": np.nan,
        }
    )

comparacion_modelos = (
    pd.DataFrame(filas_modelos)
    .sort_values(["RMSE", "MAE"])
    .reset_index(drop=True)
)

```

```

display(
    comparacion_modelos.style.format(
        {"MAE": "{:,.0f}", "RMSE": "{:,.0f}", "AIC": "{:.1f}", "BIC": "{:.1f}"}
    )
)

mejor_modelo_total = comparacion_modelos.iloc[0]["Modelo"]
print(f"Modelo final seleccionado por menor RMSE: {mejor_modelo_total}")
print(
    f"MAE={comparacion_modelos.iloc[0]['MAE']:.0f}; "
    f"RMSE={comparacion_modelos.iloc[0]['RMSE']:.0f} viajeros."
)

fig, ax = plt.subplots(figsize=(16, 7))
ax.plot(total_train.index[-36:], total_train.iloc[-36:], color="black",
        label="Entrenamiento")
ax.plot(total_test, color="navy", lw=2, label="Observado (prueba)")
for nombre, pron in predicciones.items():
    ax.plot(pron, lw=1.4, alpha=.82, label=nombre)
ax.axvline(total_test.index[0], color="black", ls="--")
ax.set(title="Predicciones fuera de muestra: serie obligatoria",
        xlabel="Fecha", ylabel="Viajeros")
ax.legend(ncol=2)
plt.show()

print(
    "Conclusión predictiva: el error debe interpretarse frente al volumen
    mensual "
    f"promedio de prueba ({total_test.mean():.0f}). El RMSE del modelo elegido
    "
    f"equivale a {100*comparacion_modelos.iloc[0]['RMSE']/total_test.mean():.
    1f}% "
    "de ese promedio; los choques y cambios de nivel posteriores a 2020 limitan
    "
    "la extrapolación basada solo en historia."
)

```

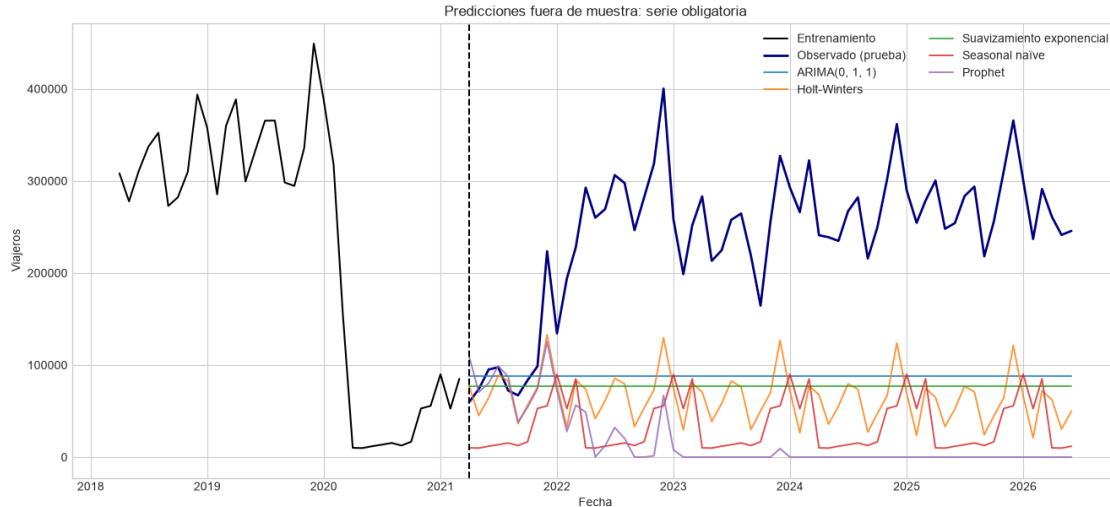
22:34:19 - cmdstanpy - INFO - Chain [1] start processing

22:34:19 - cmdstanpy - INFO - Chain [1] done processing

<pandas.io.formats.style.Styler at 0x12e78c3b0>

Modelo final seleccionado por menor RMSE: ARIMA(0, 1, 1)

MAE=156,000; RMSE=170,738 viajeros.



Conclusión predictiva: el error debe interpretarse frente al volumen mensual promedio de prueba (241,617). El RMSE del modelo elegido equivale a 70.7% de ese promedio; los choques y cambios de nivel posteriores a 2020 limitan la extrapolación basada solo en historia.

3 3b-3c. Categorías seleccionadas: País y Región dos

Se seleccionan las categorías **país de residencia** y **Región dos**. En Región dos, el Top 3 se determina en 2009-2026. Para País se usa la ventana común 2009-2022: la inspección muestra que Guatemala deja de aparecer como país desde 2023 durante la transición a agrupaciones de mercado. Región dos sustituye a Vía porque la serie marítima comparable deja de tener observaciones positivas después de 2016 y no permite evaluar pronósticos en 2021-2026.

```
[13]: top3_paises = (
    df_ts[df_ts["Año"] <= 2022]
    .groupby("País")["Viajero"].sum().nlargest(3).index.tolist()
)
top3_regiones = (
    df_ts.groupby("Región dos")["Viajero"].sum().nlargest(3).index.tolist()
)

series_paises = {
    pais: serie_mensual(df_ts, "País", pais).loc[:"2022-12-01"]
    for pais in top3_paises
}
series_regiones = {
    region: serie_mensual(df_ts, "Región dos", region)
    for region in top3_regiones
}
```

```

print("Top 3 países por total acumulado:", ", ".join(top3_paises))
print("Top 3 regiones por total acumulado:", ", ".join(top3_regiones))
display(
    pd.DataFrame(
        {
            "Categoría": ["País"] * 3 + ["Región dos"] * 3,
            "Serie": top3_paises + top3_regiones,
            "Inicio": [s.index.min() for s in series_paises.values()]
+ [s.index.min() for s in series_regiones.values()],
            "Fin": [s.index.max() for s in series_paises.values()]
+ [s.index.max() for s in series_regiones.values()],
            "Frecuencia": [s.index.freqstr for s in series_paises.values()]
+ [s.index.freqstr for s in series_regiones.values()],
            "Observaciones": [len(s) for s in series_paises.values()]
+ [len(s) for s in series_regiones.values()],
        }
    )
)

fig, axes = plt.subplots(2, 1, figsize=(16, 10), sharex=True)
for nombre, serie in series_paises.items():
    axes[0].plot(serie, label=nombre)
axes[0].set_title("Top 3 países: series mensuales")
axes[0].set_ylabel("Viajeros")
axes[0].legend()
for nombre, serie in series_regiones.items():
    axes[1].plot(serie, label=nombre)
axes[1].set_title("Top 3 de Región dos: series mensuales")
axes[1].set_ylabel("Viajeros")
axes[1].legend()
plt.tight_layout()
plt.show()

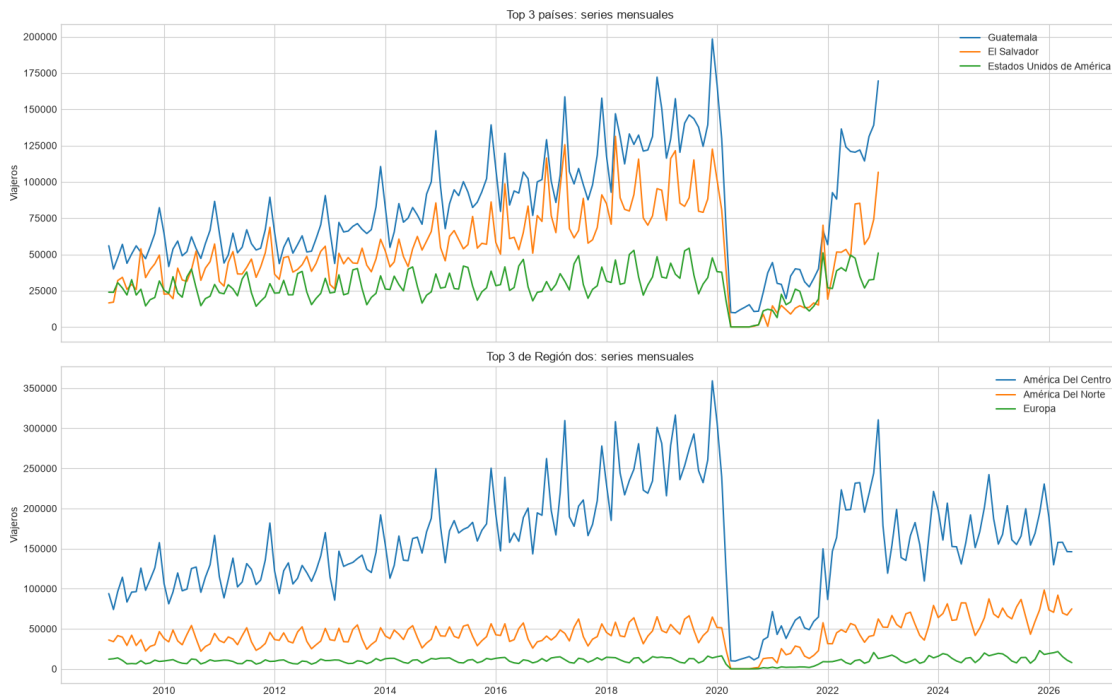
```

Top 3 países por total acumulado: Guatemala, El Salvador, Estados Unidos de América

Top 3 regiones por total acumulado: América Del Centro, América Del Norte, Europa

	Categoría	Serie	Inicio	Fin	Frecuencia	\
0	País	Guatemala	2009-01-01	2022-12-01	MS	
1	País	El Salvador	2009-01-01	2022-12-01	MS	
2	País	Estados Unidos de América	2009-01-01	2022-12-01	MS	
3	Región dos	América Del Centro	2009-01-01	2026-06-01	MS	
4	Región dos	América Del Norte	2009-01-01	2026-06-01	MS	
5	Región dos	Europa	2009-01-01	2026-06-01	MS	
	Observaciones					
0		168				

1	168
2	168
3	210
4	210
5	210



4 4. Análisis completo de las categorías seleccionadas

A continuación se aplica el mismo procedimiento 4a-4k a las seis series seleccionadas: los tres países y las tres regiones con mayor acumulado. Cada serie se divide temporalmente 70/30, se descompone, se contrasta con ADF, se examinan ACF/PACF, se prueban varios ARIMA y se comparan contra Prophet, Holt-Winters, suavizamiento exponencial y seasonal naïve.

Para que la comparación sea reproducible, el ARIMA debe tener residuos compatibles con ruido blanco (Ljung-Box $p > 0.05$) y, entre los que cumplen, se elige el menor AIC/BIC. Después compite contra los modelos alternativos mediante MAE y RMSE fuera de muestra.

```
[14]: import logging
logging.getLogger("cmdstanpy").setLevel(logging.WARNING)

def evaluar_serie_categoria(serie, nombre, categoria):
    train, test = dividir_serie(serie)

    medias = train.rolling(12).mean().dropna()
    varianzas = train.rolling(12).var().dropna()
```

```

corr_media_var = medias.corr(varianzas)
requiere_transformacion = bool(
    np.isfinite(corr_media_var) and corr_media_var > 0.5
)

# Se modela log1p en todas las series para reducir la influencia de
# 2020 y mantener pronósticos positivos; la necesidad estricta de
# transformación se informa por separado.
train_modelo = np.log1p(train)
adf_tabla, estacionaria = tabla_adf(train_modelo)
d = int(adf_tabla.iloc[-1]["d"])

stl = STL(np.log1p(train), period=12, robust=True).fit()
fuerza_estacional = max(
    0.0,
    1 - np.var(stl.resid) / np.var(stl.seasonal + stl.resid),
)

candidatos = [
    (p, d, q)
    for p, q in product(range(6), range(6))
    if not (p == 0 and q == 0)
]
filas_arima = []
ajustes = {}
pronosticos_arima = {}

for orden in candidatos:
    try:
        ajuste = ARIMA(
            train_modelo,
            order=orden,
            enforce_stationarity=False,
            enforce_invertibility=False,
        ).fit()
        pron_log = ajuste.forecast(steps=len(test))
        pron = pd.Series(
            np.maximum(0, np.exp1(np.asarray(pron_log))),
            index=test.index,
        )
        lb_p = acorr_ljungbox(
            ajuste.resid.dropna(), lags=[12], return_df=True
        )["lb_pvalue"].iloc[0]
        filas_arima.append(
            {
                "Categoría": categoria,
                "Serie": nombre,
            }
        )
    except:
        pass

```

```

        "Modelo": f"ARIMA{orden}",
        "Orden": orden,
        "AIC": ajuste.aic,
        "BIC": ajuste.bic,
        "Ljung_Box_p12": lb_p,
        "MAE": mean_absolute_error(test, pron),
        "RMSE": np.sqrt(mean_squared_error(test, pron)),
    }
)
ajustes[orden] = ajuste
pronosticos_arima[orden] = pron
except Exception:
    continue

tabla_arima_local = (
    pd.DataFrame(filas_arima).sort_values("AIC").reset_index(drop=True)
)
if tabla_arima_local.empty:
    raise RuntimeError(f"No fue posible ajustar ARIMA para {nombre}")

tabla_arima_local["Residuos_blanco_5pct"] = (
    tabla_arima_local["Ljung_Box_p12"] > 0.05
)
validos = tabla_arima_local[tabla_arima_local["Residuos_blanco_5pct"]]
if validos.empty:
    mejor_fila_arima = tabla_arima_local.iloc[0]
    criterio_arima = "menor AIC; ningún candidato pasó Ljung-Box"
else:
    mejor_fila_arima = validos.sort_values(["BIC", "AIC"]).iloc[0]
    criterio_arima = "menor BIC entre candidatos con residuos blancos"
mejor_orden = mejor_fila_arima["Orden"]
mejor_ajuste = ajustes[mejor_orden]
predicciones_locales = {
    mejor_fila_arima["Modelo"]: pronosticos_arima[mejor_orden]
}

# Holt-Winters
hw = ExponentialSmoothing(
    train,
    trend="add",
    damped_trend=True,
    seasonal="add",
    seasonal_periods=12,
    initialization_method="estimated",
).fit(optimized=True)
predicciones_locales["Holt-Winters"] = hw.forecast(len(test)).clip(lower=0)

```

```

# Suavizamiento exponencial simple
ses = SimpleExpSmoothing(
    train, initialization_method="estimated"
).fit(optimized=True)
predicciones_locales["Suavizamiento exponencial"] = (
    ses.forecast(len(test)).clip(lower=0)
)

# Seasonal naïve
predicciones_locales["Seasonal naïve"] = pd.Series(
    np.resize(train.iloc[-12:].to_numpy(), len(test)),
    index=test.index,
)

# Prophet
prophet_df = train.rename_axis("ds").reset_index(name="y")
prophet = Prophet(
    yearly_seasonality=True,
    weekly_seasonality=False,
    daily_seasonality=False,
    seasonality_mode="additive",
    changepoint_prior_scale=0.10,
)
prophet.fit(prophet_df)
prophet_fcst = prophet.predict(pd.DataFrame({"ds": test.index}))
predicciones_locales["Prophet"] = pd.Series(
    np.maximum(0, prophet_fcst["yhat"].to_numpy()),
    index=test.index,
)

filas_modelos = []
for modelo, pron in predicciones_locales.items():
    es_arima = modelo == mejor_fila_arima["Modelo"]
    filas_modelos.append(
        {
            "Categoría": categoria,
            "Serie": nombre,
            "Modelo": modelo,
            "MAE": mean_absolute_error(test, pron),
            "RMSE": np.sqrt(mean_squared_error(test, pron)),
            "AIC": mejor_fila_arima["AIC"] if es_arima else np.nan,
            "BIC": mejor_fila_arima["BIC"] if es_arima else np.nan,
        }
    )
tabla_modelos_local = (
    pd.DataFrame(filas_modelos)
    .sort_values(["RMSE", "MAE"])

```

```

        .reset_index(drop=True)
    )

    return {
        "categoria": categoria,
        "nombre": nombre,
        "serie": serie,
        "train": train,
        "test": test,
        "corr_media_var": corr_media_var,
        "requiere_transformacion": requiere_transformacion,
        "stl": stl,
        "fuerza_estacional": fuerza_estacional,
        "adf": adf_tabla,
        "d": d,
        "estacionaria": estacionaria,
        "tabla_arima": tabla_arima_local,
        "mejor_fila_arima": mejor_fila_arima,
        "criterio_arima": criterio_arima,
        "mejor_orden_arima": mejor_orden,
        "mejor_ajuste_arima": mejor_ajuste,
        "predicciones": predicciones_locales,
        "tabla_modelos": tabla_modelos_local,
    }

resultados_categorias = {}
for categoria, diccionario in [
    ("País", series_paises),
    ("Región dos", series_regiones),
]:
    for nombre, serie in diccionario.items():
        print(f"Modelando {categoria}: {nombre}...")
        resultados_categorias[(categoria, nombre)] = evaluarSerieCategoria(
            serie, nombre, categoria
        )

print("Modelado completo de las seis series.")

```

Modelando País: Guatemala...

Modelando País: El Salvador...

Modelando País: Estados Unidos de América...

Modelando Región dos: América Del Centro...

Modelando Región dos: América Del Norte...

Modelando Región dos: Europa...

Modelado completo de las seis series.

4.1 4a-4e. Inicio, fin, frecuencia, componentes y estacionariedad

La tabla resume la evidencia de cada serie. “Transformación necesaria” responde específicamente al patrón media-varianza; independientemente de esa decisión, los ARIMA se estiman con log1p como protección frente a valores extremos y para mantener pronósticos no negativos.

```
[15]: filas_diagnostico = []
for resultado in resultados_categorias.values():
    adf_final = resultado["adf"].iloc[-1]
    filas_diagnostico.append(
        {
            "Categoría": resultado["categoria"],
            "Serie": resultado["nombre"],
            "Inicio": resultado["serie"].index.min(),
            "Fin": resultado["serie"].index.max(),
            "Frecuencia": resultado["serie"].index.freqstr,
            "n_train": len(resultado["train"]),
            "n_test": len(resultado["test"]),
            "Corr. media-varianza": resultado["corr_media_var"],
            "Transformación necesaria": resultado["requiere_transformacion"],
            "Fuerza estacional": resultado["fuerza_estacional"],
            "d": resultado["d"],
            "ADF final": adf_final["ADF"],
            "ADF p-value": adf_final["p_value"],
        }
    )

diagnostico_series = pd.DataFrame(filas_diagnostico)
display(
    diagnostico_series.style.format(
        {
            "Corr. media-varianza": "{:.3f}",
            "Fuerza estacional": "{:.3f}",
            "ADF final": "{:.3f}",
            "ADF p-value": "{:.4f}",
        }
    )
)

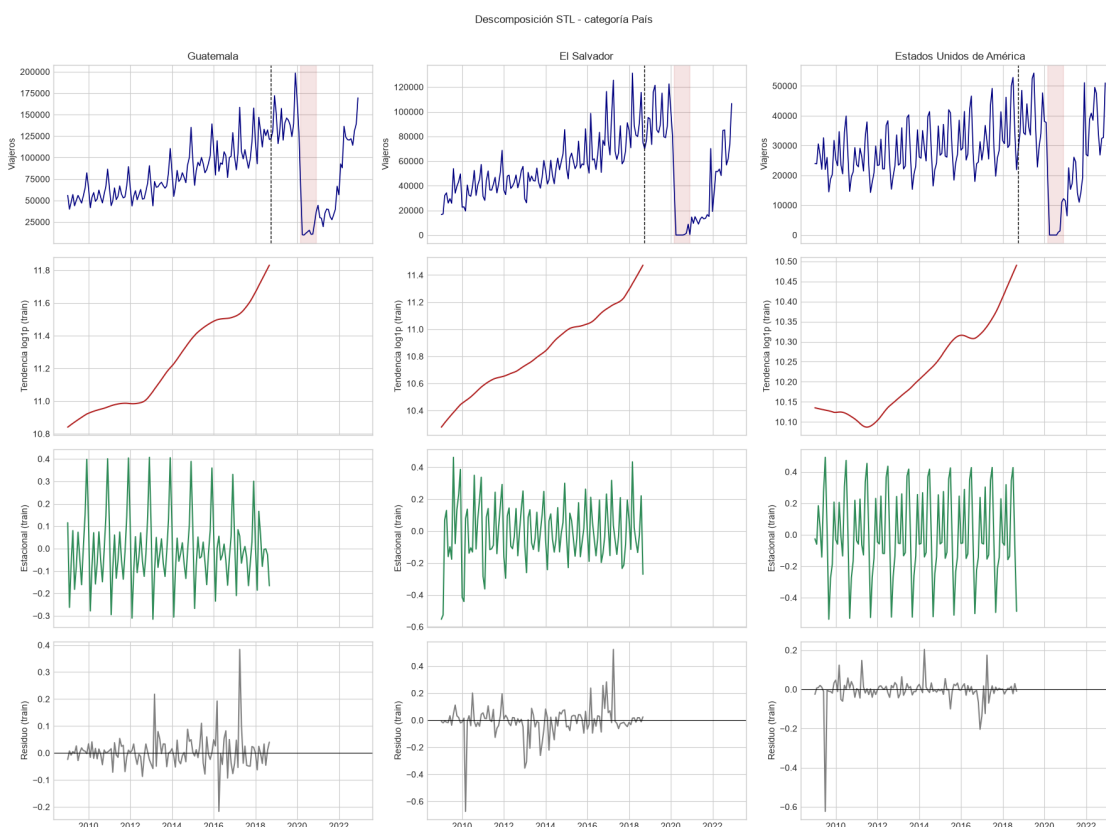
for categoria in ["País", "Región dos"]:
    resultados_cat = [
        r for r in resultados_categorias.values()
        if r["categoria"] == categoria
    ]
    fig, axes = plt.subplots(4, 3, figsize=(18, 13), sharex="col")
    for col, r in enumerate(resultados_cat):
        axes[0, col].plot(r["serie"], color="navy", lw=1.2)
        axes[0, col].axvspan(
```

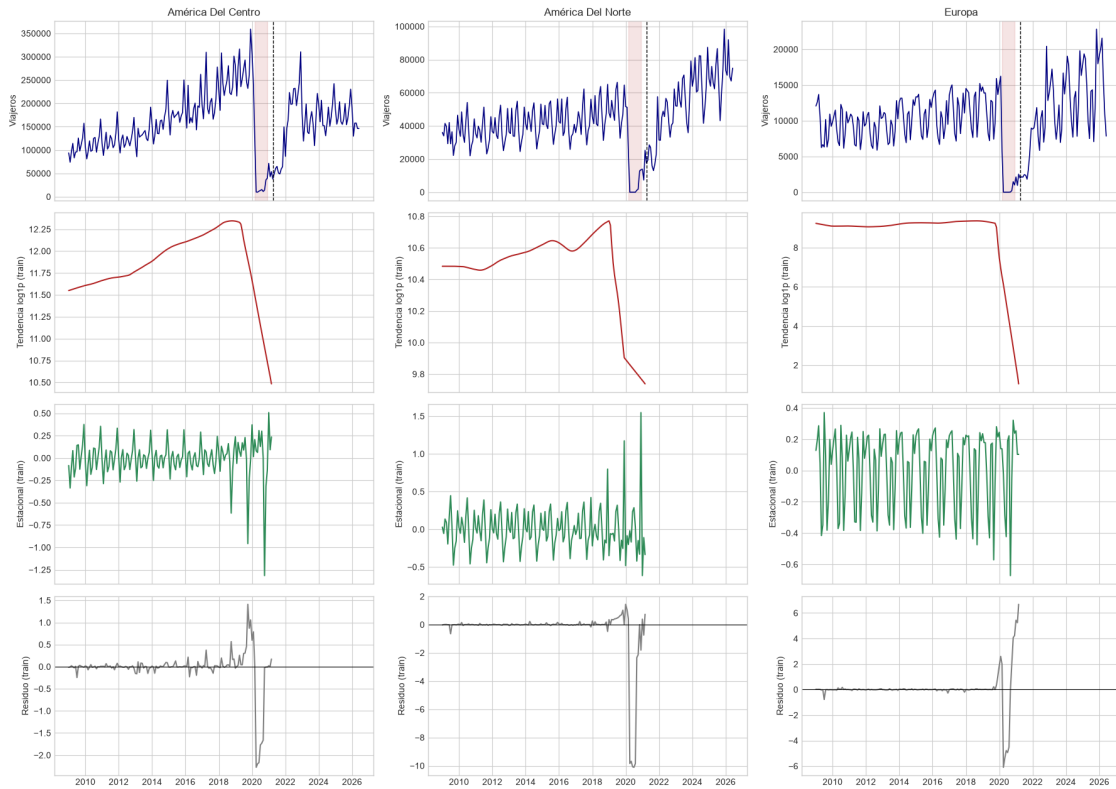
```

pd.Timestamp("2020-03-01"), pd.Timestamp("2020-12-01"),
color="firebrick", alpha=.12
)
axes[0, col].axvline(
    r["test"].index[0], color="black", ls="--", lw=1
)
axes[0, col].set_title(r["nombre"])
axes[0, col].set_ylabel("Viajeros")
axes[1, col].plot(r["stl"].trend, color="firebrick")
axes[1, col].set_ylabel("Tendencia log1p (train)")
axes[2, col].plot(r["stl"].seasonal, color="seagreen")
axes[2, col].set_ylabel("Estacional (train)")
axes[3, col].plot(r["stl"].resid, color="gray")
axes[3, col].axhline(0, color="black", lw=.8)
axes[3, col].set_ylabel("Residuo (train)")
fig.suptitle(f"Descomposición STL - categoría {categoria}", y=1.01)
plt.tight_layout()
plt.show()

```

<pandas.io.formats.style.Styler at 0x12e0aa2d0>





```
[16]: print("INTERPRETACIÓN INDIVIDUAL DE LAS SERIES")
for r in resultados_categorias.values():
    serie = r["serie"]
    impacto = 100 * (serie.loc["2020"].sum() / serie.loc["2019"].sum() - 1)
    anio_post = "2022" if r["categoria"] == "País" else "2025"
    recuperacion = 100 * (
        serie.loc[anio_post].sum() / serie.loc["2019"].sum() - 1
    )
    fuerza = r["fuerza_estacional"]
    intensidad = (
        "débil" if fuerza < 0.30
        else "moderada" if fuerza < 0.60
        else "fuerte"
    )
    implicacion_estacional = (
        "los cambios de nivel dominan sobre el calendario"
        if fuerza < 0.30
        else "el calendario aporta información, aunque no domina completamente"
        if fuerza < 0.60
        else "el patrón mensual es una fuente importante de variación"
    )
```

```

pre = np.log1p(serie.loc[:"2019-12-01"])
pendiente_pre = linregress(np.arange(len(pre)), pre.to_numpy()).slope
crecimiento_pre = 100 * (np.exp(12 * pendiente_pre) - 1)
direccion = "creciente" if pendiente_pre > 0 else "decreciente"
transformacion = (
    "sí, la varianza aumenta con el nivel"
    if r["requiere_transformacion"]
    else "no es estrictamente necesaria según media-varianza; log1p se usa_
    por robustez"
)
adf_final = r["adf"].iloc[-1]
print(f"\n{r['categoria']} - {r['nombre']}")
print(
    f"- Primera vista: tendencia pre-pandemia {direccion} "
    f"({crecimiento_pre:+.1f}% anual aproximado); impacto 2020 vs 2019="
    f"{impacto:+.1f}% y nivel {anio_post} vs 2019={recuperacion:+.1f}%."
)
if r["categoria"] == "País":
    print(
        " Se usa 2022 para recuperación porque desde 2023 inicia la "
        "transición de País a agrupaciones de mercado."
    )
print(
    f"- Estacionalidad {intensidad} (fuerza={fuerza:.3f}); "
    f"{implicacion_estacional}."
)
print(f"- Transformación: {transformacion}.")
print(
    f"- Estacionariedad en media: ADF final p={adf_final['p_value']:.4f} "
    f"después de d={r['d']}; con p<0.05 se rechaza raíz unitaria."
)

```

INTERPRETACIÓN INDIVIDUAL DE LAS SERIES

País - Guatemala

- Primera vista: tendencia pre-pandemia creciente (+10.9% anual aproximado); impacto 2020 vs 2019=-70.2% y nivel 2022 vs 2019=-16.9%.

Se usa 2022 para recuperación porque desde 2023 inicia la transición de País a agrupaciones de mercado.

- Estacionalidad fuerte (fuerza=0.860); el patrón mensual es una fuente importante de variación.

- Transformación: sí, la varianza aumenta con el nivel.

- Estacionariedad en media: ADF final p=0.0008 después de d=1; con p<0.05 se rechaza raíz unitaria.

País - El Salvador

- Primera vista: tendencia pre-pandemia creciente (+12.0% anual aproximado); impacto 2020 vs 2019=-79.8% y nivel 2022 vs 2019=-36.5%.

Se usa 2022 para recuperación porque desde 2023 inicia la transición de País a agrupaciones de mercado.

- Estacionalidad fuerte (fuerza=0.733); el patrón mensual es una fuente importante de variación.
- Transformación: sí, la varianza aumenta con el nivel.
- Estacionariedad en media: ADF final $p=0.0000$ después de $d=1$; con $p<0.05$ se rechaza raíz unitaria.

País - Estados Unidos de América

- Primera vista: tendencia pre-pandemia creciente (+4.3% anual aproximado); impacto 2020 vs 2019=-74.0% y nivel 2022 vs 2019=-2.7%.

Se usa 2022 para recuperación porque desde 2023 inicia la transición de País a agrupaciones de mercado.

- Estacionalidad fuerte (fuerza=0.918); el patrón mensual es una fuente importante de variación.
- Transformación: sí, la varianza aumenta con el nivel.
- Estacionariedad en media: ADF final $p=0.0000$ después de $d=1$; con $p<0.05$ se rechaza raíz unitaria.

Región dos - América Del Centro

- Primera vista: tendencia pre-pandemia creciente (+10.3% anual aproximado); impacto 2020 vs 2019=-74.4% y nivel 2025 vs 2019=-34.0%.
- Estacionalidad débil (fuerza=0.004); los cambios de nivel dominan sobre el calendario.
- Transformación: no es estrictamente necesaria según media-varianza; log1p se usa por robustez.
- Estacionariedad en media: ADF final $p=0.0358$ después de $d=1$; con $p<0.05$ se rechaza raíz unitaria.

Región dos - América Del Norte

- Primera vista: tendencia pre-pandemia creciente (+3.4% anual aproximado); impacto 2020 vs 2019=-74.1% y nivel 2025 vs 2019=+39.3%.
- Estacionalidad débil (fuerza=0.000); los cambios de nivel dominan sobre el calendario.
- Transformación: no es estrictamente necesaria según media-varianza; log1p se usa por robustez.
- Estacionariedad en media: ADF final $p=0.0489$ después de $d=1$; con $p<0.05$ se rechaza raíz unitaria.

Región dos - Europa

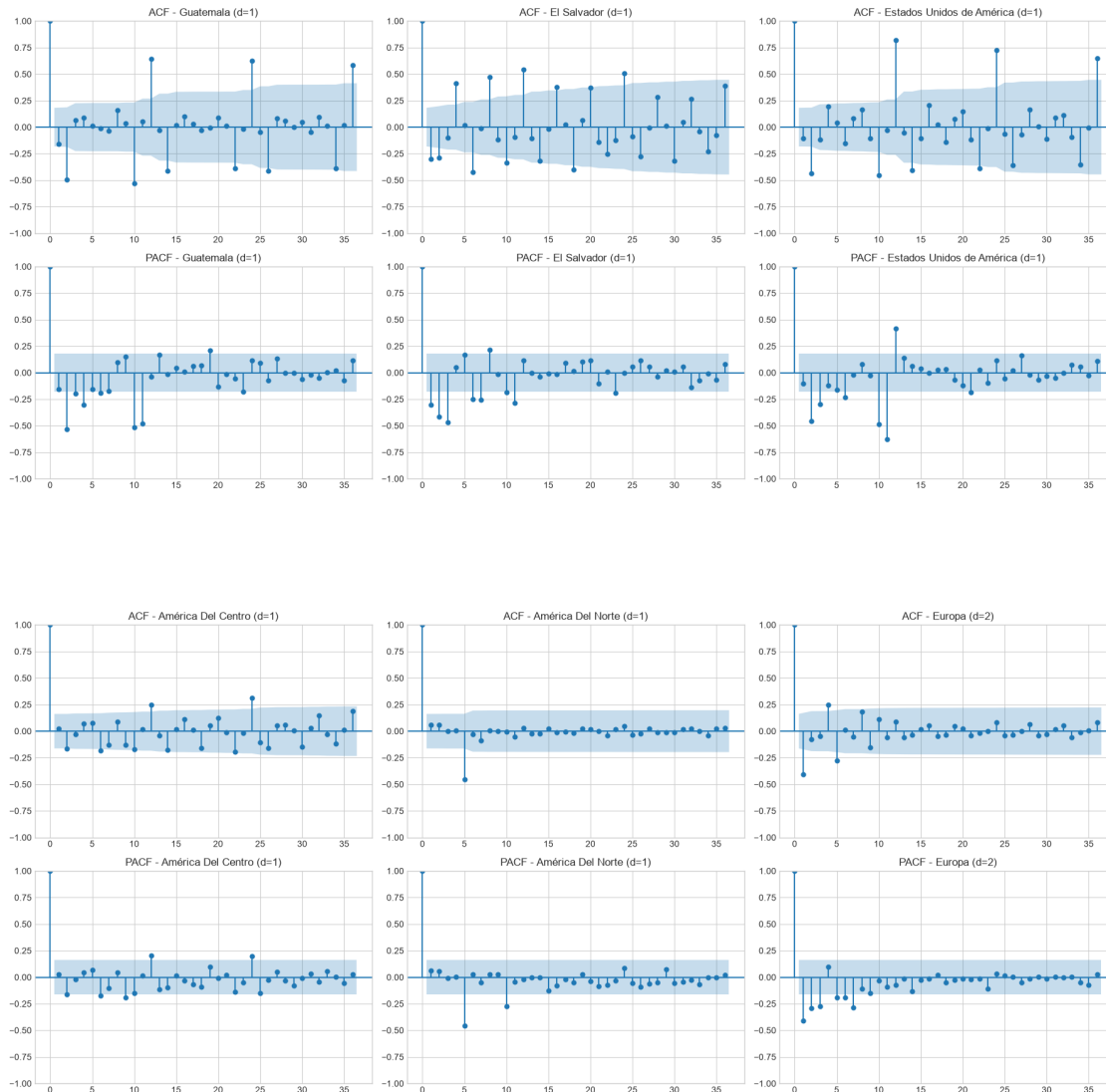
- Primera vista: tendencia pre-pandemia creciente (+2.9% anual aproximado); impacto 2020 vs 2019=-71.8% y nivel 2025 vs 2019=+24.8%.
- Estacionalidad débil (fuerza=0.075); los cambios de nivel dominan sobre el calendario.
- Transformación: no es estrictamente necesaria según media-varianza; log1p se usa por robustez.
- Estacionariedad en media: ADF final $p=0.0000$ después de $d=2$; con $p<0.05$ se rechaza raíz unitaria.

4.2 4e-4f. ACF/PACF y parámetros ARIMA

d es la mínima diferenciación que logra ADF $p < 0.05$. Después de esa diferenciación se inspeccionan ACF y PACF. Se contrastan p, q entre 0 y 5 y BIC penaliza los órdenes innecesariamente altos. El orden final es el menor BIC entre los candidatos cuyos residuos pasan Ljung-Box.

```
[17]: for categoria in ["País", "Región dos"]:
    resultados_cat = [
        r for r in resultados_categorias.values()
        if r["categoria"] == categoria
    ]
    fig, axes = plt.subplots(2, 3, figsize=(18, 8))
    for col, r in enumerate(resultados_cat):
        plot_acf(r["estacionaria"], lags=36, ax=axes[0, col])
        axes[0, col].set_title(f"ACF - {r['nombre']} (d={r['d']})")
        plot_pacf(
            r["estacionaria"], lags=36, ax=axes[1, col], method="ywm"
        )
        axes[1, col].set_title(f"PACF - {r['nombre']} (d={r['d']})")
    plt.tight_layout()
    plt.show()

tablas_visibles = []
for r in resultados_categorias.values():
    visibles = pd.concat(
        [
            r["tabla_arima"].head(8),
            r["mejor_fila_arima"].to_frame().T,
        ],
        ignore_index=True,
    ).drop_duplicates(subset=["Categoría", "Serie", "Modelo"])
    tablas_visibles.append(visibles)
tabla_candidatos_arima = pd.concat(
    tablas_visibles, ignore_index=True
).drop(columns="Orden")
display(
    tabla_candidatos_arima.style.format(
        {
            "AIC": "{:.1f}",
            "BIC": "{:.1f}",
            "Ljung_Box_p12": "{:.3f}",
            "MAE": "{:,.0f}",
            "RMSE": "{:,.0f}",
        }
    )
)
```



<pandas.io.formats.style.Styler at 0x12e26c860>

4.3 4g. Residuos del mejor ARIMA

Un residuo adecuado debe oscilar alrededor de cero y no conservar autocorrelación. La prueba Ljung-Box al rezago 12 se reporta junto con AIC y BIC; $p > 0.05$ indica que no se detecta autocorrelación residual anual.

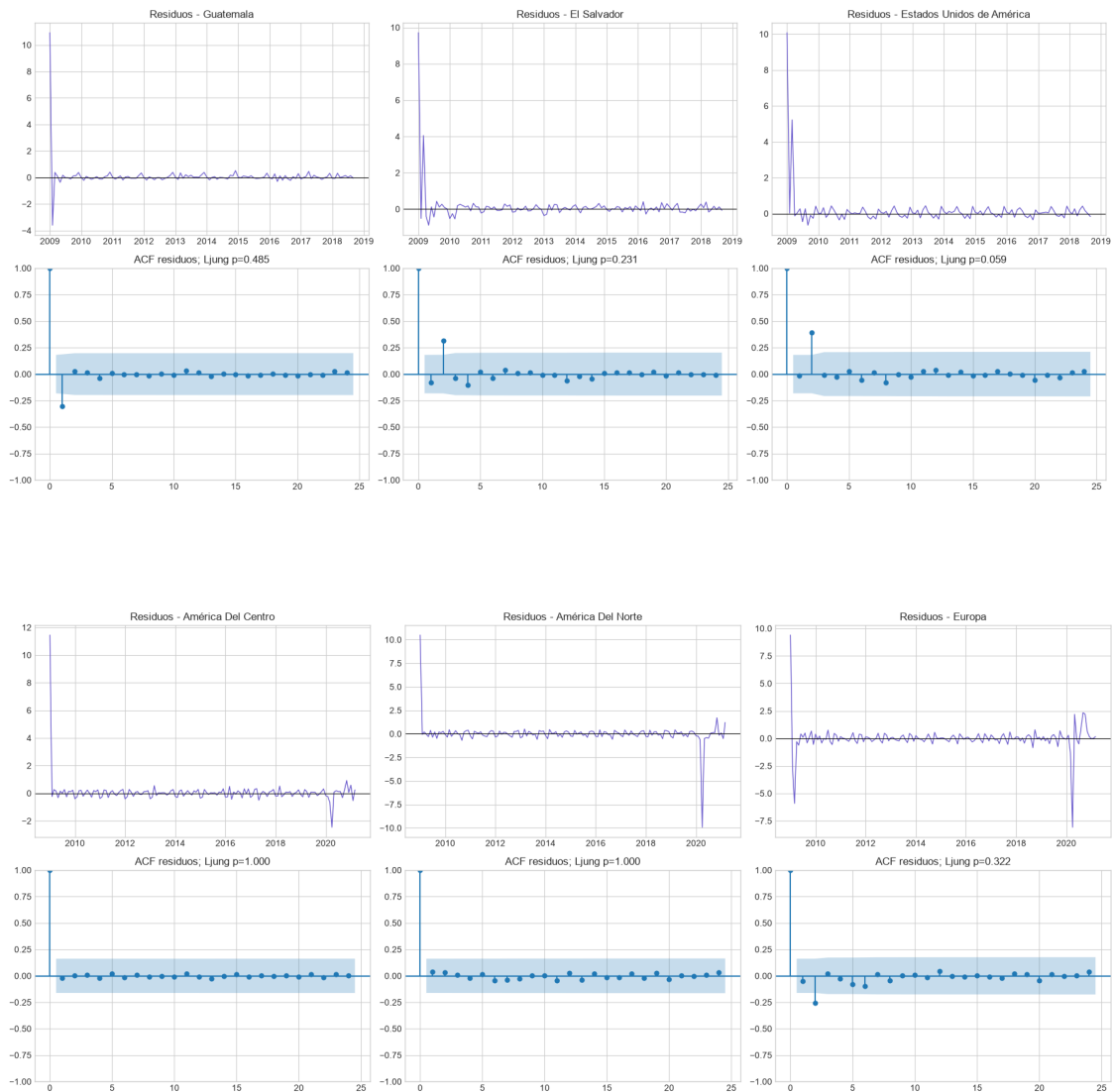
```
[18]: filas_mejor_arima = []
for categoria in ["País", "Región dos"]:
    resultados_cat = [
        r for r in resultados_categorias.values()
        if r["categoria"] == categoria
    ]
```

```

fig, axes = plt.subplots(2, 3, figsize=(18, 8))
for col, r in enumerate(resultados_cat):
    mejor = r["mejor_fila_arima"]
    residuos = r["mejor_ajuste_arima"].resid.dropna()
    axes[0, col].plot(residuos, color="slateblue", lw=1)
    axes[0, col].axhline(0, color="black", lw=.8)
    axes[0, col].set_title(f"Residuos - {r['nombre']}")
    plot_acf(residuos, lags=24, ax=axes[1, col])
    axes[1, col].set_title(
        f"ACF residuos; Ljung p={mejor['Ljung_Box_p12']:.3f}"
    )
    filas_mejor_arima.append(
        {
            "Categoría": categoria,
            "Serie": r["nombre"],
            "Mejor ARIMA": mejor["Modelo"],
            "AIC": mejor["AIC"],
            "BIC": mejor["BIC"],
            "Ljung-Box p(12)": mejor["Ljung_Box_p12"],
        }
    )
plt.tight_layout()
plt.show()

resumen_mejor_arima = pd.DataFrame(filas_mejor_arima)
display(
    resumen_mejor_arima.style.format(
        {
            "AIC": "{:.1f}",
            "BIC": "{:.1f}",
            "Ljung-Box p(12)": "{:.3f}",
        }
    )
)

```



<pandas.io.formats.style.Styler at 0x138371f10>

```
[19]: print("JUSTIFICACIÓN DE p, d, q Y RESIDUOS")
for r in resultados_categorias.values():
    mejor = r["mejor_fila_arima"]
    p_sel, d_sel, q_sel = mejor["Orden"]
    estado_residuos = (
        "no se detecta autocorrelación residual"
        if mejor["Ljung_Box_p12"] > 0.05
        else "persiste autocorrelación; el ARIMA es una aproximación imperfecta"
    )
    print(f"\n{r['categoria']} - {r['nombre']}: {mejor['Modelo']}")
    print(
        f"- d={d_sel} es la mínima diferencia requerida por ADF; "
```

```

    f"p={p_sel} y q={q_sel} se contrastaron a partir de los primeros "
    "rezagos visibles en PACF y ACF."
)
print(
    f"- Criterio final: {r['criterio_arima']}; "
    f"AIC={mejor['AIC']:.1f}, BIC={mejor['BIC']:.1f}."
)
print(
    f"- Ljung-Box p(12)={mejor['Ljung_Box_p12']:.3f}: {estado_residuos}."
)

```

JUSTIFICACIÓN DE p, d, q Y RESIDUOS

País - Guatemala: ARIMA(1, 1, 3)

- d=1 es la mínima diferencia requerida por ADF; p=1 y q=3 se contrastaron a partir de los primeros rezagos visibles en PACF y ACF.
- Criterio final: menor BIC entre candidatos con residuos blancos; AIC=-81.2, BIC=-67.6.
- Ljung-Box p(12)=0.485: no se detecta autocorrelación residual.

País - El Salvador: ARIMA(4, 1, 3)

- d=1 es la mínima diferencia requerida por ADF; p=4 y q=3 se contrastaron a partir de los primeros rezagos visibles en PACF y ACF.
- Criterio final: menor BIC entre candidatos con residuos blancos; AIC=-46.8, BIC=-25.1.
- Ljung-Box p(12)=0.231: no se detecta autocorrelación residual.

País - Estados Unidos de América: ARIMA(2, 1, 4)

- d=1 es la mínima diferencia requerida por ADF; p=2 y q=4 se contrastaron a partir de los primeros rezagos visibles en PACF y ACF.
- Criterio final: menor BIC entre candidatos con residuos blancos; AIC=-6.7, BIC=12.3.
- Ljung-Box p(12)=0.059: no se detecta autocorrelación residual.

Región dos - América Del Centro: ARIMA(0, 1, 1)

- d=1 es la mínima diferencia requerida por ADF; p=0 y q=1 se contrastaron a partir de los primeros rezagos visibles en PACF y ACF.
- Criterio final: menor BIC entre candidatos con residuos blancos; AIC=83.5, BIC=89.4.
- Ljung-Box p(12)=1.000: no se detecta autocorrelación residual.

Región dos - América Del Norte: ARIMA(0, 1, 5)

- d=1 es la mínima diferencia requerida por ADF; p=0 y q=5 se contrastaron a partir de los primeros rezagos visibles en PACF y ACF.
- Criterio final: menor BIC entre candidatos con residuos blancos; AIC=383.4, BIC=401.1.
- Ljung-Box p(12)=1.000: no se detecta autocorrelación residual.

Región dos - Europa: ARIMA(1, 2, 3)

- d=2 es la mínima diferencia requerida por ADF; p=1 y q=3 se contrastaron a partir de los primeros rezagos visibles en PACF y ACF.
- Criterio final: menor BIC entre candidatos con residuos blancos; AIC=360.5, BIC=375.2.
- Ljung-Box p(12)=0.322: no se detecta autocorrelación residual.

4.4 4h-4k. Modelos alternativos, predicción y selección final

AIC/BIC se muestran solamente para ARIMA. La comparación entre familias se realiza fuera de muestra con MAE y RMSE. El mejor modelo final de cada serie es el de menor RMSE.

```
[20]: comparacion_modelos_categorias = pd.concat(
    [r["tabla_modelos"] for r in resultados_categorias.values()],
    ignore_index=True,
)
display(
    comparacion_modelos_categorias.style.format(
        {
            "MAE": "{:,.0f}",
            "RMSE": "{:,.0f}",
            "AIC": "{:.1f}",
            "BIC": "{:.1f}",
        }
    )
)

filas_finales = []
for categoria in ["País", "Región dos"]:
    resultados_cat = [
        r for r in resultados_categorias.values()
        if r["categoria"] == categoria
    ]
    fig, axes = plt.subplots(1, 3, figsize=(19, 5))
    for ax, r in zip(axes, resultados_cat):
        mejor = r["tabla_modelos"].iloc[0]
        mejor_nombre = mejor["Modelo"]
        ax.plot(r["test"], color="navy", lw=2, label="Observado")
        ax.plot(
            r["predicciones"][mejor_nombre],
            color="firebrick", lw=1.8, label=mejor_nombre
        )
        ax.set_title(
            f"{r['nombre']}\nRMSE={mejor['RMSE']:.0f}; MAE={mejor['MAE']:.0f}"
        )
        ax.legend()
    filas_finales.append(
        {
```

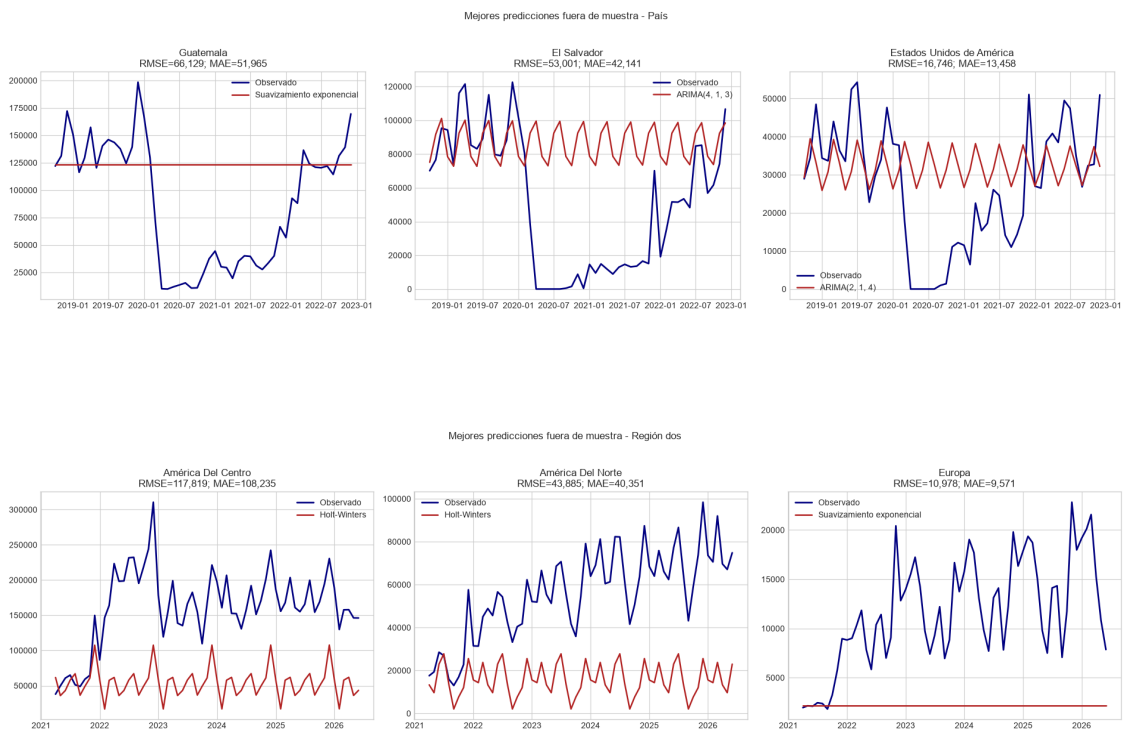
```

        "Categoría": categoria,
        "Serie": r["nombre"],
        "Modelo final": mejor_nombre,
        "MAE": mejor["MAE"],
        "RMSE": mejor["RMSE"],
        "RMSE / promedio prueba (%)":
            100 * mejor["RMSE"] / r["test"].mean(),
    }
)
fig.suptitle(f"Mejores predicciones fuera de muestra - {categoria}", y=1.03)
plt.tight_layout()
plt.show()

resumen_modelos_categorias = pd.DataFrame(filas_finales)
display(
    resumen_modelos_categorias.style.format(
        {
            "MAE": "{:,.0f}",
            "RMSE": "{:,.0f}",
            "RMSE / promedio prueba (%)": "{:.1f}%",
        }
    )
)

```

<pandas.io.formats.style.Styler at 0x11d48d0a0>



<pandas.io.formats.style.Styler at 0x12e611f10>

5 5. Análisis comparativo con evidencia estadística

Para País y Región dos se comparan cuatro indicadores:

1. **Estacionalidad:** fuerza STL (0 = débil, 1 = fuerte).
2. **Crecimiento:** pendiente anualizada de la tendencia STL antes de la pandemia (2009-2019).
3. **Volatilidad:** desviación estándar del cambio logarítmico mensual.
4. **Impacto COVID-19:** variación del total 2020 respecto de 2019.

La ventana pre-pandemia separa crecimiento estructural del choque de 2020. El análisis comparativo utiliza el período completo porque aquí el objetivo es describir, no seleccionar modelos. Para País, estacionalidad y volatilidad se limitan a 2009-2022, última ventana común antes de la transición observada desde 2023.

La descomposición diagnóstica del inciso 4 usa únicamente entrenamiento y puede mostrar estacionalidad pre-pandemia fuerte. Aquí la fuerza se recalcula sobre toda la ventana comparable, incluyendo el quiebre de 2020; por eso mide qué tan estable sobrevivió el patrón estacional al cambio estructural y puede resultar considerablemente menor.

```
[21]: def metricas_comparativas(diccionario, categoria):
    filas = []
    for nombre, serie in diccionario.items():
        # País se limita a 2022 por la transición de cobertura observada
        # desde 2023; Región dos conserva toda la ventana.
        serie_comparable = (
            serie.loc["2022-12-01"] if categoria == "País" else serie
        )
        log_serie = np.log1p(serie_comparable)
        stl = STL(log_serie, period=12, robust=True).fit()
        fuerza = max(
            0.0,
            1 - np.var(stl.resid) / np.var(stl.seasonal + stl.resid),
        )

        pre = stl.trend.loc["2019-12-01"].dropna()
        x = np.arange(len(pre))
        reg = linregress(x, pre.to_numpy())
        # En log escala, 12*pendiente aproxima crecimiento porcentual anual.
        crecimiento_anual_pct = 100 * (np.exp(12 * reg.slope) - 1)

        cambios_log = np.diff(np.log1p(serie_comparable.to_numpy()))
        volatilidad = 100 * np.std(cambios_log, ddof=1)

        total_2019 = serie.loc["2019"].sum()
        total_2020 = serie.loc["2020"].sum()
```

```

    impacto = 100 * (total_2020 / total_2019 - 1)

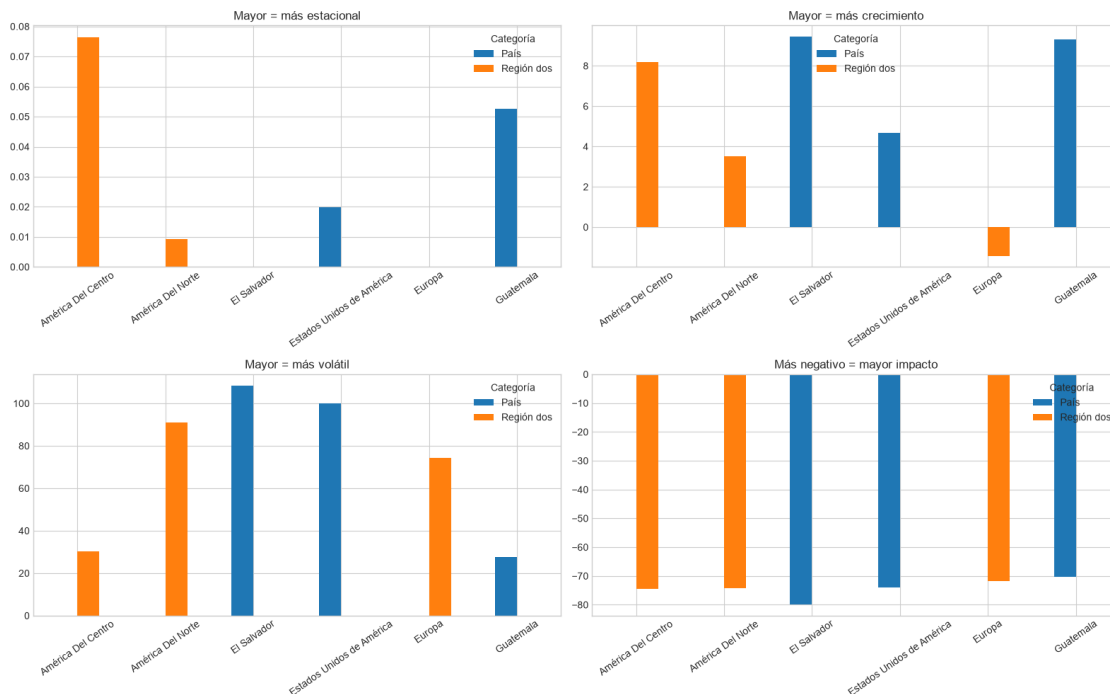
    filas.append(
        {
            "Categoría": categoria,
            "Serie": nombre,
            "Fuerza estacional": fuerza,
            "Crecimiento anual pre-COVID (%)": crecimiento_anual_pct,
            "Volatilidad mensual log (%)": volatilidad,
            "Impacto 2020 vs 2019 (%)": impacto,
        }
    )
    return pd.DataFrame(filas)

comparativo = pd.concat(
    [
        metricas_comparativas(series_paises, "País"),
        metricas_comparativas(series_regiones, "Región dos"),
    ],
    ignore_index=True,
)
display(
    comparativo.style.format(
        {
            "Fuerza estacional": "{:.3f}",
            "Crecimiento anual pre-COVID (%)": "{:+.2f}%",
            "Volatilidad mensual log (%)": "{:.2f}%",
            "Impacto 2020 vs 2019 (%)": "{:+.1f}%",
        }
    )
)

fig, axes = plt.subplots(2, 2, figsize=(16, 10))
indicadores = [
    ("Fuerza estacional", "Mayor = más estacional"),
    ("Crecimiento anual pre-COVID (%)", "Mayor = más crecimiento"),
    ("Volatilidad mensual log (%)", "Mayor = más volátil"),
    ("Impacto 2020 vs 2019 (%)", "Más negativo = mayor impacto"),
]
for ax, (columna, titulo) in zip(axes.flat, indicadores):
    piv = comparativo.pivot(index="Serie", columns="Categoría", values=columna)
    piv.plot(kind="bar", ax=ax)
    ax.set_title(titulo)
    ax.set_xlabel("")
    ax.tick_params(axis="x", rotation=35)
plt.tight_layout()
plt.show()

```

<pandas.io.formats.style.Styler at 0x12e159070>



```
[22]: def responder_categoria(tabla, categoria):
    t = tabla[tabla["Categoría"] == categoria].set_index("Serie")
    mayor_est = t["Fuerza estacional"].idxmax()
    mayor_crec = t["Crecimiento anual pre-COVID (%)"].idxmax()
    mayor_vol = t["Volatilidad mensual log (%)"].idxmax()
    mayor_covid = t["Impacto 2020 vs 2019 (%)"].idxmin()
    print(f"\n{n{categoria.upper()}}")
    print(
        f"1. Mayor estacionalidad: {mayor_est} "
        f"(fuerza={t.loc[mayor_est, 'Fuerza estacional']:.3f})."
    )
    print(
        f"2. Mayor tendencia de crecimiento pre-COVID: {mayor_crec} "
        f"({t.loc[mayor_crec, 'Crecimiento anual pre-COVID (%)']:.2f}% anual)."
    )
    print(
        f"3. Mayor volatilidad: {mayor_vol} "
        f"({t.loc[mayor_vol, 'Volatilidad mensual log (%)']:.2f}% mensual)."
    )
    print(
        f"4. Más afectada por la pandemia: {mayor_covid} "
        f"({t.loc[mayor_covid, 'Impacto 2020 vs 2019 (%)']:.1f}% en 2020 vs_
↪ 2019)."
```

```

    )
    if t["Fuerza estacional"].max() < 0.10:
        print(
            "    Nota: aunque esa serie es la mayor dentro de la categoría, "
            "todas muestran fuerza estacional absoluta débil (<0.10)."
        )

responder_categoria(comparativo, "País")
responder_categoria(comparativo, "Región dos")

```

PAÍS

1. Mayor estacionalidad: Guatemala (fuerza=0.053).
 2. Mayor tendencia de crecimiento pre-COVID: El Salvador (+9.45% anual).
 3. Mayor volatilidad: El Salvador (108.29% mensual).
 4. Más afectada por la pandemia: El Salvador (-79.8% en 2020 vs 2019).
- Nota: aunque esa serie es la mayor dentro de la categoría, todas muestran fuerza estacional absoluta débil (<0.10).

REGIÓN DOS

1. Mayor estacionalidad: América Del Centro (fuerza=0.076).
 2. Mayor tendencia de crecimiento pre-COVID: América Del Centro (+8.20% anual).
 3. Mayor volatilidad: América Del Norte (90.93% mensual).
 4. Más afectada por la pandemia: América Del Centro (-74.4% en 2020 vs 2019).
- Nota: aunque esa serie es la mayor dentro de la categoría, todas muestran fuerza estacional absoluta débil (<0.10).

5.1 5b. Descubrimientos útiles para INGUAT

- La magnitud de la caída de 2020 no debe tratarse como ruido ordinario: es un cambio estructural. Para planificación conviene usar modelos que admitan cambios de nivel y actualizarlos a medida que entran datos.
- La fuerza estacional estimada es baja en términos absolutos. INGUAT no debería suponer picos mensuales estables sin revalidarlos por mercado y región; los cambios de nivel pesan más que el calendario en estas series.
- La región o el país con mayor crecimiento pre-COVID señala dónde existía expansión estructural antes del choque, mientras que la recuperación posterior permite verificar si ese patrón regresó.
- La serie más volátil requiere escenarios e intervalos de predicción amplios, no una única cifra presupuestaria.
- Para decisiones oficiales debe conservarse el filtro Turista + Excursionista. Mezclar la categoría Viajero introduciría una caída artificial por el cambio de definición entre 2022 y 2023.
- Desde 2023 se observa la transición de País a agrupaciones de mercado; por ello cualquier estrategia a nivel de país debe validar primero la comparabilidad semántica con la fuente.